



Laboratorio Integración

Estudiantes:

Carol Araya Conejo - Carné 2024089174

Helena Vargas Quirós – Carné 2024145198

Sede: Campus Tecnológico Central Cartago

Curso: Aseguramiento de la Calidad del Software

Profesora

Alicia Salazar

Tabla de contenidos

Matriz de casos de prueba	3
Métricas	8
Pruebas de integración realizadas con Postman	9
Consulta de productos o servicios	9
Agregar un producto al servicio	10
Consultar un producto del catálogo por id	12
Agregar producto al carrito	13
Obtener un carrito por id	15
Consultar y validar total	17
Pruebas negativas obligatorias	18
Enviar cantidad cero o negativa.	19
Consultar un producto inexistente	20
Enviar tipo de dato incorrecto.	22
Enviar body incompleto a /products	24
Enviar body incompleto a /carts	25
Validar errores claros y controlados.	27
Ejecución de colección completa	28
Pruebas de integración realizadas con Pytest	31
Consultar API pública y validar contrato	31
Insertar producto consultado en SQLite	32
Validar total del carrito	32
Validar cantidades	34
Ingresar productos inexistentes	36
Medir tiempo de respuesta (< 1000 ms)	37
Prueba de contrato estricta	38
Idempotencia	39
Manejo de errores	41
Prueba de integración parcial	42
Mock opcional	43
Comparación Postman y Pytest	45
Reporte Final	46
Conclusiones	54

Matriz de casos de prueba

En el presente laboratorio, la matriz fue diseñada con el propósito de documentar los casos de prueba de integración asociados al flujo funcional seleccionado, el cual incluye la consulta de productos, la creación de carritos, el cálculo del total y la validación de errores entre otros.

ID	Caso	Objetivo	Endpoint	Datos de entrada	Pasos	Resultado esperado	Validación	Métrica
TC-1	Consultar catálogo	Verificar que el API retorna correctamente la lista de productos	GET /products	No aplica	1. Enviar request GET a /products	200 OK, lista no vacía, cada producto con id, title, price, description, category e image	Estado HTTP + contrato JSON	Tiempo de respuesta < 1000 ms
TC-2	Añadir un producto nuevo	Evaluar la capacidad del sistema para recibir un nuevo producto y registrar correctamente sus datos	POST/products	{ "title": "string", "price": 0.1, "description": "string", "category": "string", "image": "http://example.com" }	1. Guardar los datos del producto en variables. 2. Ejecutar POST /products. 3. Validar respuesta.	201 CREATED. Solicita obligatoriamente el nombre, precio, descripción, categoría e imagen y lo añade correctamente al catálogo de productos. El id es generado automáticamente. El nuevo producto debe verse reflejado en el catálogo.	Contrato + Estado HTTP	Tiempo de respuesta menor a 1000 ms.

ID	Caso	Objetivo	Endpoint	Datos de entrada	Pasos	Resultado esperado	Validación	Métrica
		dentro del catálogo.						
TC-3	Consultar un producto	Evaluar la capacidad del sistema para retornar correctamente la información de un producto existente en el catálogo.	GET /products / {id}	id: 1	1. Guardar id en variables globales 2. Enviar request GET a /products/{id}	200 OK. El producto debe tener su id, nombre, precio, descripción, categoría e imagen. El producto debe estar en existencia.	Contrato + Estado HTTP	Tiempo de respuesta < 1000 ms
TC-4	Añadir nuevo carrito	Verificar que se pueda crear un carrito usando un producto consultado previamente	POST/ carts	userId y products con id del producto	1. Ejecutar GET /products. 2. Guardar los datos del primer producto en variables. 3. Ejecutar POST /carts. 4. Validar respuesta.	201 Created. Retorna un carrito con id, userId y arreglo products.	Estado HTTP + contrato JSON	Tiempo de respuesta menor a 1000 ms. Si la API pública no tiene carrito real, se permite simularlo con un endpoint POST de prueba y documentar la limitación.
TC-5	Obtener un	Obtener	GET/cart	id, este por medio de	1. Guardar id	200 OK. Retorna	Estado HTTP +	Tiempo de

ID	Caso	Objetivo	Endpoint	Datos de entrada	Pasos	Resultado esperado	Validación	Métrica
	carrito	detalles de un carrito por medio del ID	s/{id}	parametro en el EndPoint	en variables globales 2. Enviar request GET a /carts/{id}	una estructura de carrito con su ID, id user, productos.	contrato JSON	respuesta < 1000 ms
TC-6	Consultar y validar total del carrito	Validar el total esperado usando el precio del producto consultado y la cantidad agregada al carrito	POST /carts	{ "id": 0, "userId": 0, "products": [{ "id": 0, "title": "string", "price": 0.1, "description": "string", "category": "string", "image": "http://example.com" }] }	1. Ejecutar GET /products. 2. Guardar precio. 3. Crear carrito. 4. Calcular total manualmente.	200 OK. Total = precio del producto × cantidad.	Regla de negocio	Exactitud del cálculo
TC-7	Enviar cantidad inválida (simulada)	Validar que el API maneje datos inválidos en el carrito	POST /carts	Producto inválido (id negativo o vacío)	1. Enviar request con producto inválido.	400 Bad Request. Se debe recibir un error por parte del API	Manejo de errores	Tasa de error
TC-8	Consultar un producto inexistente	Verificar si la API valida	GET /products / {id}	id: -10	1. Enviar request con id de	400 Bad Request. No devolverá el producto, sino que	Contrato + Estado HTTP	Tasa de error

ID	Caso	Objetivo	Endpoint	Datos de entrada	Pasos	Resultado esperado	Validación	Métrica
		correctamente la consulta de productos inexistentes			producto inexistente	devolverá un mensaje de error.		
TC-9	Enviar tipo de dato incorrecto	Verificar si la API valida correctamente los tipos de datos del request	POST /carts	Tipos incorrectos (string, number, etc.)	1. Enviar request con tipos incorrectos. 2. Analizar respuesta.	400 Bad Request	Contrato + Estado HTTP Manejo de errores	Tasa de error
TC-10	Enviar un body incompleto a /products.	Verificar si la API valida correctamente el body incompleto en /products	POST/products	{ "id": 1, "title": "string" }	1. Enviar request con body incompleto. 2. Analizar respuesta.	400 Bad Request. No se insertará el producto, y se devolverá un mensaje de error.	Contrato + Estado HTTP	Tasa de error
TC-11	Enviar un body incompleto a /carts.	Verificar si la API valida correctamente el	POST/carts	{ "id": 0 }	1. Enviar request con body incompleto. 2. Analizar	400 Bad Request.	Contrato + Estado HTTP	Tasa de error

ID	Caso	Objetivo	Endpoint	Datos de entrada	Pasos	Resultado esperado	Validación	Métrica
		body incompleto en /carts			respuesta.			
TC-12	Prueba de integración parcial	Validar qué ocurre si la API responde correctamente, pero la base de datos falla al guardar la información	GET /products	Producto obtenido desde GET /products		La API responde 200, pero SQLite genera un error controlado	Manejo de errores + persistencia	Defecto controlado / tolerancia a fallos

Métricas

En este laboratorio, las métricas se calcularon a partir de los resultados obtenidos en Postman, con el fin de analizar la estabilidad del sistema y respaldar las conclusiones con datos objetivos.

Métrica	Cálculo	Criterio
Porcentaje de casos aprobados	Casos aprobados / total de casos x 100	$\geq 85\%$
Tiempo promedio de respuesta	Promedio de ms por endpoint crítico	Por ejemplo < 1000 ms
Defectos encontrados	Cantidad de errores funcionales, técnicos o de contrato	Registrar severidad
Cobertura de endpoints	Endpoints probados / endpoints del flujo x 100	$\geq 80\%$ del flujo elegido
Validaciones por endpoint	Cantidad de assertions por request	Mínimo 3 por endpoint importante
Tasa de error	Respuestas 4xx/5xx inesperadas / total de ejecuciones	Lo más baja posible

Pruebas de integración realizadas con Postman

En el presente laboratorio se llevaron a cabo pruebas de integración utilizando la herramienta Postman sobre la API pública FakeStoreAPI. El propósito de estas pruebas fue validar la correcta interacción entre distintos endpoints dentro de un flujo funcional de consulta de productos y gestión de carritos.

Se evaluaron aspectos clave como el estado de las respuestas HTTP, la estructura de los datos (contrato JSON), el cumplimiento de reglas de negocio y el manejo de errores. Asimismo, se documentaron las limitaciones encontradas debido al uso de una API pública de prueba.

Consulta de productos o servicios

Para la validación del catálogo de productos, se utilizó el endpoint:

```
GET /products
```

Esta prueba tuvo como objetivo verificar la disponibilidad del servicio, así como la correcta estructura de los datos retornados por el API.

Validaciones realizadas

Durante la ejecución de la prueba se verificaron los siguientes aspectos:

- Código de estado HTTP igual a 200 (OK).
- Tiempo de respuesta inferior a 1000 milisegundos.
- Tipo de dato de la respuesta: arreglo (array).
- Existencia de al menos un producto en la respuesta.
- Estructura del objeto producto, verificando la presencia de los siguientes atributos:
 - id
 - title
 - price
 - description
 - category
 - image

Resultados obtenidos

El endpoint respondió correctamente en términos funcionales y estructurales; sin embargo, el tiempo de respuesta excede el criterio establecido, lo cual representa un posible riesgo en escenarios de alta demanda o uso concurrente.

Métricas asociadas

- **Porcentaje de aprobación:**

$$(4 / 5) \times 100 = 80\%$$

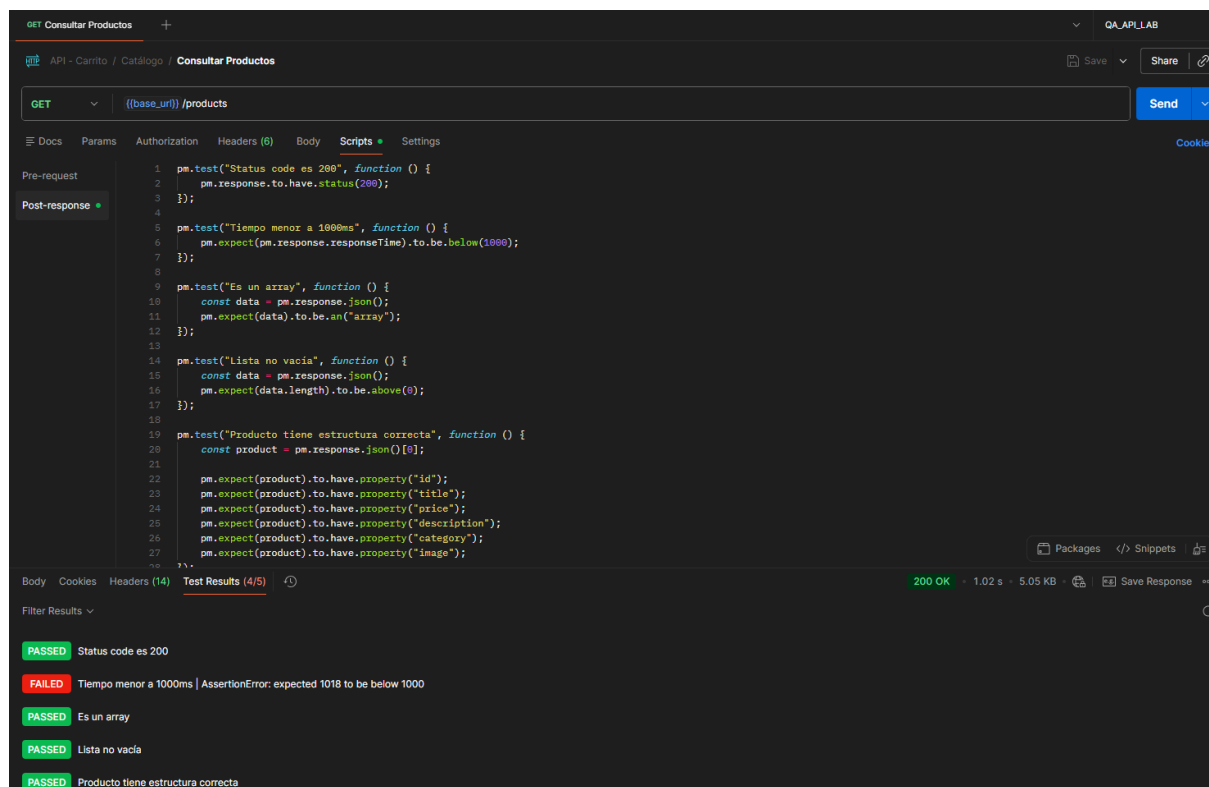
- **Tiempo de respuesta observado:** 1018 ms
- **Tiempo esperado:** < 1000 ms
- **Resultado:** Parcialmente aprobado

Agregar un producto al servicio

Para la validación del catálogo de productos, se utilizó el endpoint:

POST /products

Esta prueba tuvo como objetivo evaluar la capacidad del sistema para recibir un nuevo producto y registrar correctamente sus datos dentro del catálogo.



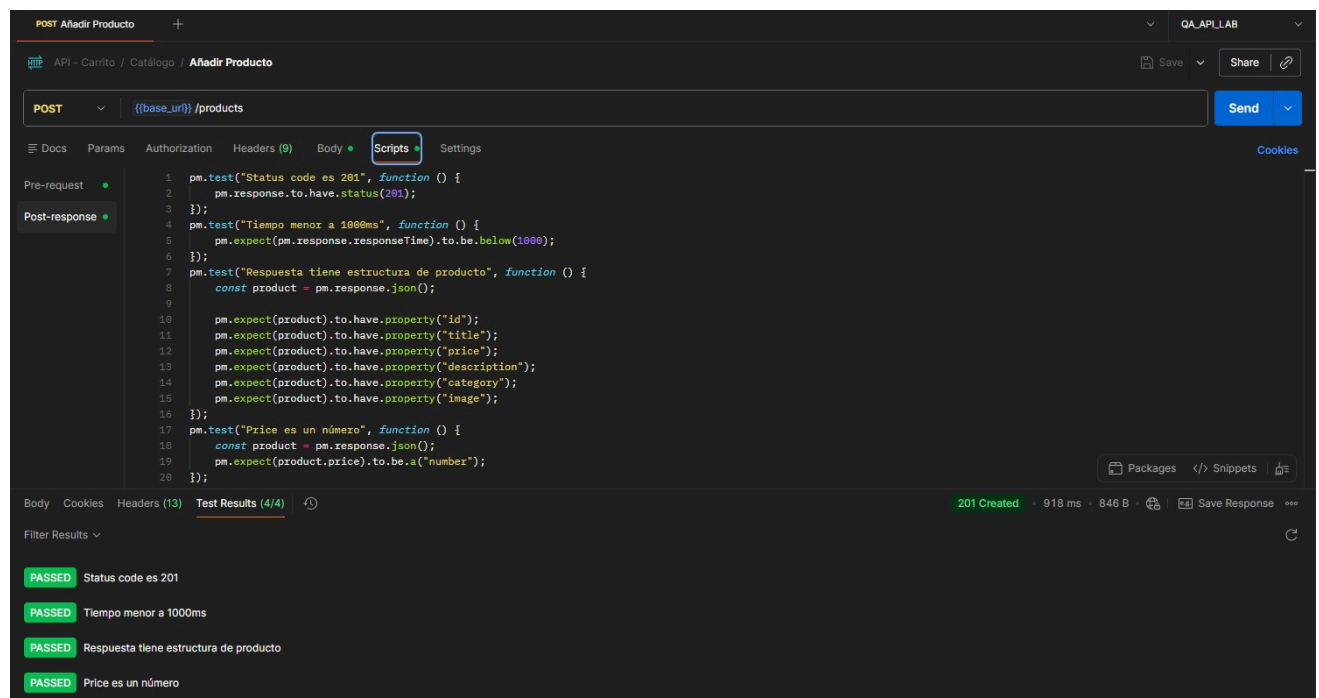
Validaciones realizadas

Durante la ejecución de la prueba se verificaron los siguientes aspectos:

- Código de estado HTTP igual a 201 (CREATED).
- Tiempo de respuesta inferior a 1000 milisegundos.
- Tipo de dato de la respuesta: objeto (JSON).
- Existencia del producto creado en la respuesta.
- Price es un número.
- Estructura del objeto producto, verificando la presencia de los siguientes atributos:
 - id
 - title
 - price
 - description
 - category
 - image

Resultados obtenidos

El endpoint respondió correctamente en términos funcionales y estructurales, ya que permitió la creación de un nuevo producto y retornó la información correspondiente.



Métricas asociadas

- **Porcentaje de aprobación:**

$$(4 / 4) \times 100 = 100\%$$
- **Tiempo de respuesta observado:** 918 ms

- **Tiempo esperado:** < 1000 ms
- **Resultado:** Aprobado

Consultar un producto del catálogo por id

Para la validación de la consulta de un producto específico, se utilizó el endpoint:

```
GET /products/{id}
```

Esta prueba tuvo como objetivo evaluar la capacidad del sistema para retornar correctamente la información de un producto existente en el catálogo.

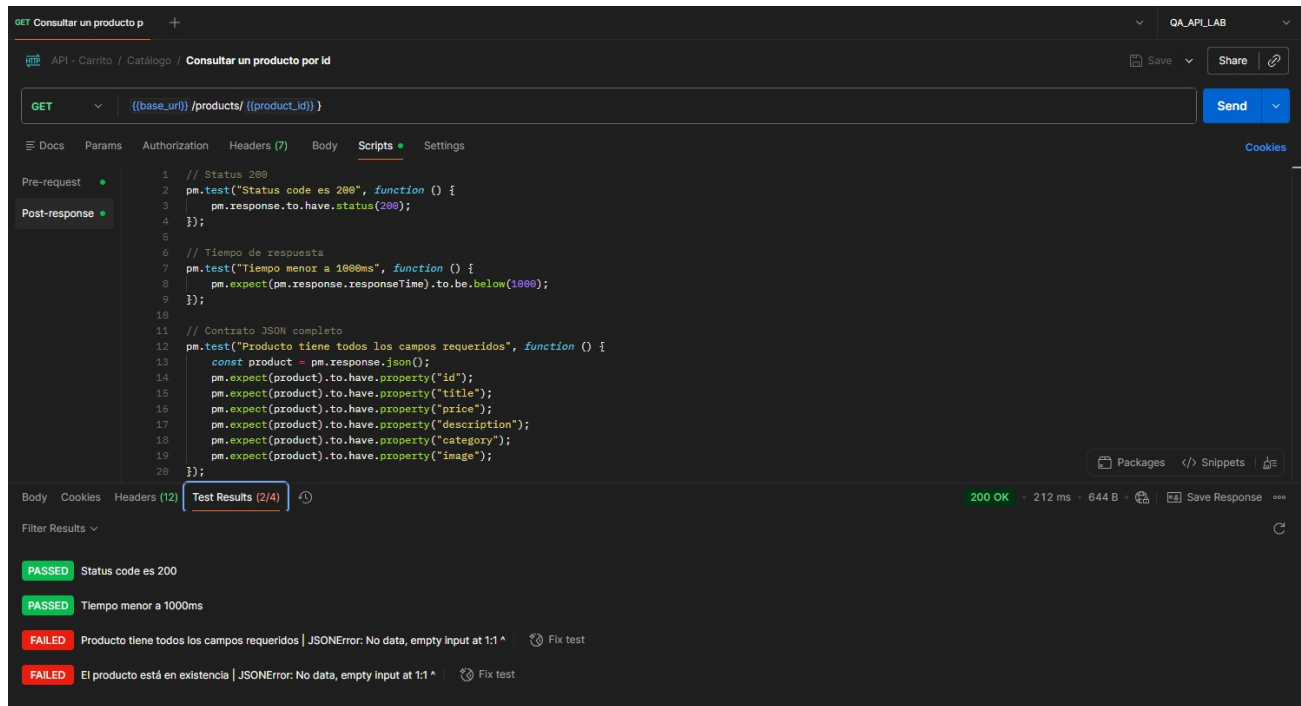
Validaciones realizadas

Durante la ejecución de la prueba se verificaron los siguientes aspectos:

- , verificando la presencia de los siguientes atributos Código de estado HTTP igual a 200 (OK).
- Tiempo de respuesta inferior a 1000 milisegundos.
- El producto retornado existe en el catálogo.
- Estructura del objeto producto
 - id
 - title
 - price
 - description
 - category
 - image

Resultados obtenidos

El endpoint respondió correctamente en términos funcionales y estructurales, retornando la información completa del producto consultado. Sin embargo, el producto no se encontraba en existencia y no todos los campos requeridos estaban presentes en la respuesta.



Métricas asociadas

- **Porcentaje de aprobación:**

$$(2 / 4) \times 100 = 50\%$$
- **Tiempo de respuesta observado:** 212 ms
- **Tiempo esperado:** < 1000 ms
- **Resultado:** Fallido

Agregar producto al carrito

Para simular la creación de un carrito de compras, se utilizó el endpoint:

POST `/carts`

Datos de entrada

Se evaluó la capacidad del sistema para recibir un producto previamente consultado y generar una estructura de carrito asociada a un usuario. Se empleó el siguiente cuerpo de la solicitud:

```

{
  "userId": 1,
  "products": [
    {

```

```
"id": {{product_id}}
}
]
}
```

Validaciones realizadas

Se verificaron los siguientes criterios:

- Código de estado HTTP igual a 201 (Created).
- Presencia de los atributos:
 - id del carrito
 - userId
 - products
- Validación de que la lista de productos no se encuentre vacía.
- Confirmación de que cada producto contiene un identificador válido.

Resultados obtenidos

El endpoint permitió la creación del carrito de manera satisfactoria. Sin embargo, la respuesta obtenida contiene únicamente el identificador del producto, sin incluir información adicional como precio, nombre o descripción.

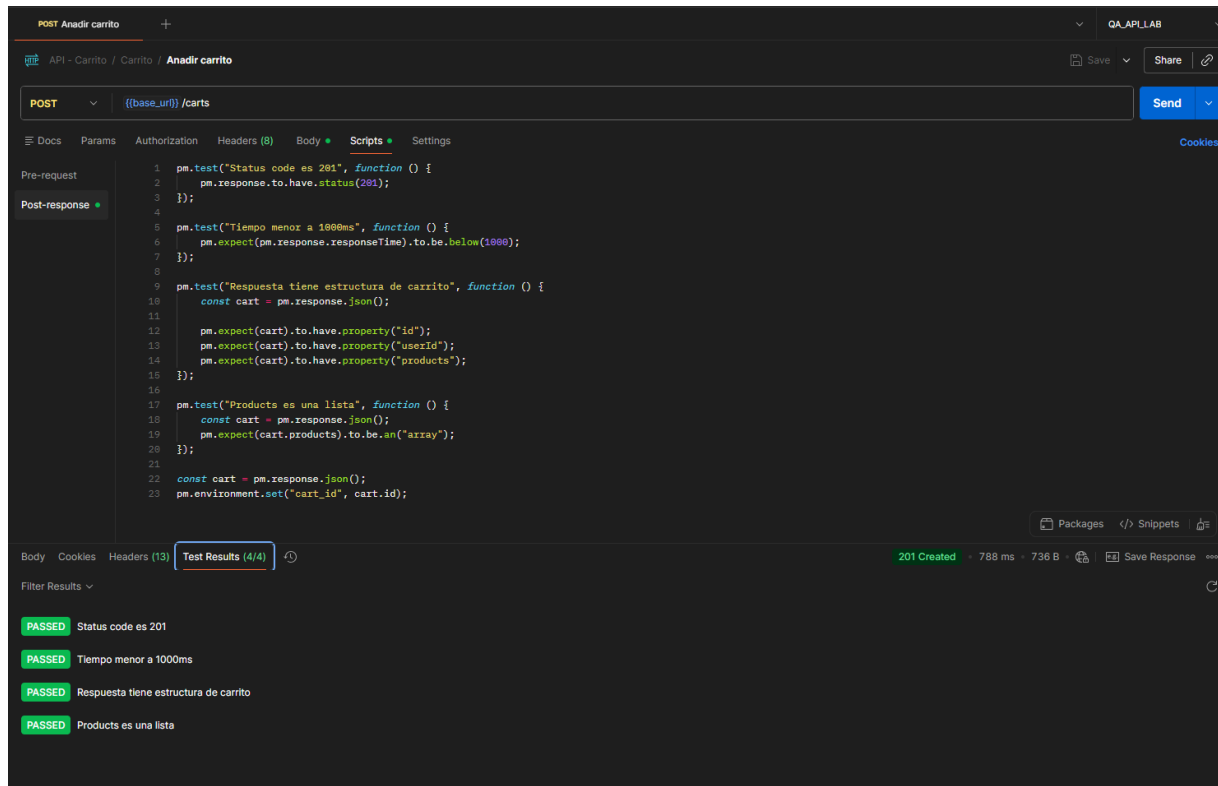
Durante la ejecución del endpoint POST /carts se obtuvieron los siguientes resultados:

- Total de validaciones ejecutadas: 4
- Validaciones aprobadas: 4
- Validaciones fallidas: 0

El endpoint respondió con código 201 (Created), indicando la correcta creación del recurso. Asimismo, el tiempo de respuesta registrado fue de 788 ms, cumpliendo con el umbral definido (< 1000 ms).

El comportamiento del servicio fue satisfactorio tanto a nivel funcional como estructural. La respuesta del API contiene los atributos esperados (id, userId, products) y cumple con las validaciones definidas en el contrato.

Adicionalmente, se verificó que la lista de productos es válida y contiene al menos un elemento, lo que confirma la correcta asociación del producto al carrito.



Métricas asociadas

- Porcentaje de aprobación:

$$(4 / 4) \times 100 = 100\%$$

- Tiempo de respuesta observado: 788 ms
- Tiempo esperado: < 1000 ms
- Resultado: Aprobado

Observaciones

A pesar del resultado exitoso, se mantiene la observación previamente identificada:

- La respuesta del API no incluye información completa del producto (como precio o descripción).
- Esto limita la validación de reglas de negocio en pruebas posteriores, como el cálculo del total del carrito.

Obtener un carrito por id

Para la validación de la consulta de un carrito específico, se utilizó el endpoint:

```
GET /carts/{id}
```

Esta prueba tuvo como objetivo evaluar la capacidad del sistema para retornar correctamente los detalles de un carrito existente, incluyendo su identificador, usuario asociado y lista de productos.

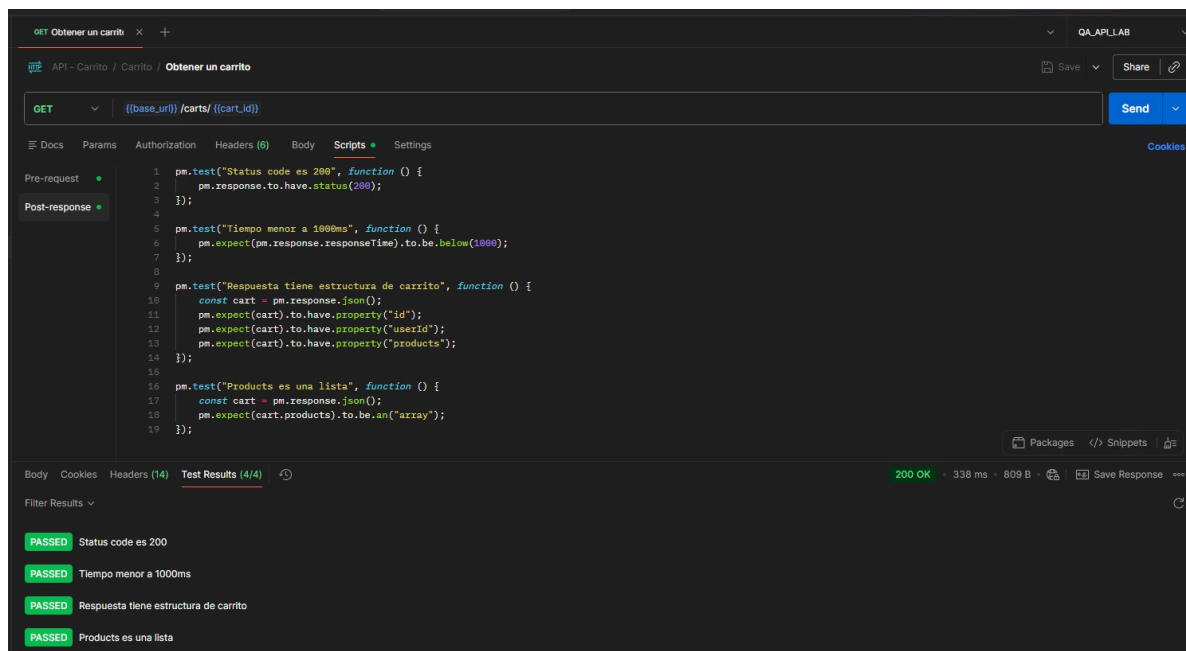
Validaciones realizadas

Durante la ejecución de la prueba se verificaron los siguientes aspectos:

- Código de estado HTTP igual a 200 (OK).
- Tiempo de respuesta inferior a 1000 milisegundos.
- El carrito retornado existe en el sistema.
- Estructura del objeto carrito, verificando la presencia de los siguientes atributos
 - id
 - userID
 - products

Resultados obtenidos

El endpoint respondió correctamente en términos funcionales y estructurales, retornando la información completa del carrito consultado. El carrito se encontraba en existencia y todos los campos requeridos estaban presentes en la respuesta.



Métricas asociadas

- Porcentaje de aprobación:

$$(4 / 4) \times 100 = 100\%$$

- **Tiempo de respuesta observado:** 338 ms
- **Tiempo esperado:** < 1000 ms
- **Resultado:** Aprobado

Consultar y validar total

El objetivo de esta prueba fue validar el cálculo del total del carrito como parte de las reglas de negocio del sistema.

Situación identificada

Ante esta limitación, se procedió a calcular el total manualmente utilizando la siguiente fórmula:

Total = precio del producto × cantidad

Para ello se utilizaron:

- El precio del producto obtenido desde el endpoint GET /products.
- Una cantidad asumida de 1 unidad.

Resultados obtenidos

El cálculo del total se realizó correctamente utilizando los datos disponibles. No obstante, no fue posible validar dicho valor contra una respuesta directa del API. La ausencia de un campo de total en la respuesta del carrito limita la validación completa de las reglas de negocio, lo cual representa una restricción significativa en el contexto de pruebas de integración.

Durante la ejecución de la prueba de cálculo del total del carrito se obtuvo el siguiente comportamiento:

- Total de validaciones ejecutadas: 1
- Validaciones aprobadas: 1
- Validaciones fallidas: 0

El endpoint respondió con código 201 (Created) y un tiempo de respuesta de 949 ms, cumpliendo con el umbral establecido (< 1000 ms).

Dado que el API no proporciona un campo de total en la respuesta del carrito, se implementó un cálculo manual utilizando los datos previamente obtenidos del endpoint GET /products.

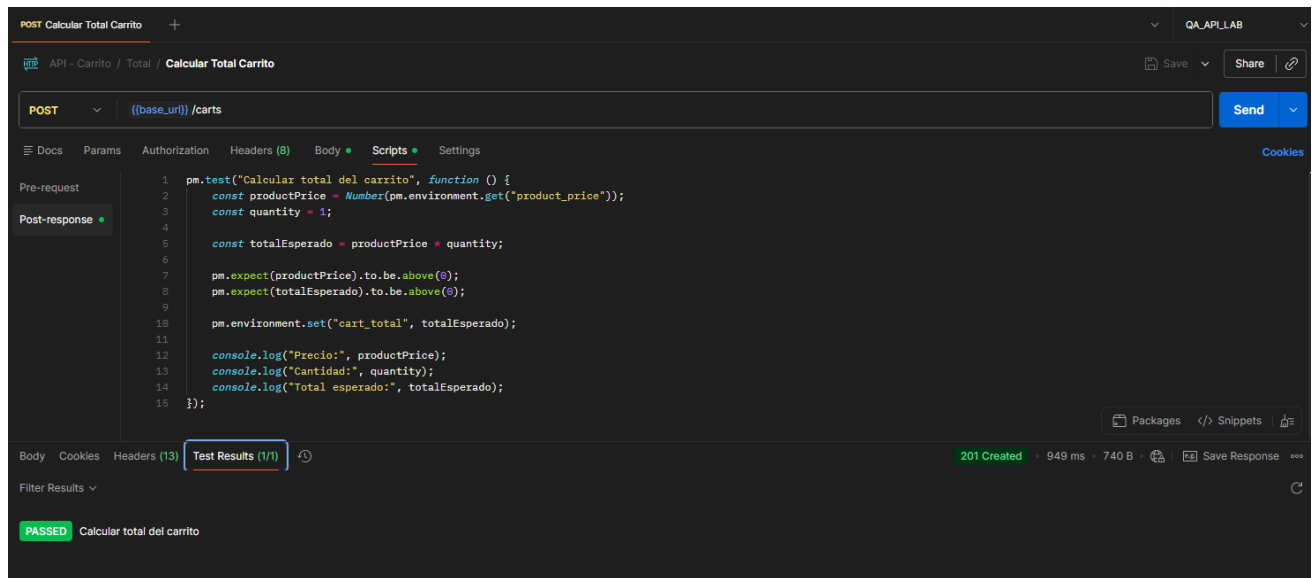
El cálculo se realizó mediante la siguiente expresión:

Total = precio del producto × cantidad

Se verificó que:

- El precio del producto es un valor válido (> 0).
- El total calculado es correcto y mayor a cero.

Este enfoque permitió validar parcialmente la regla de negocio asociada al cálculo del total del carrito.



Métricas asociadas

- Porcentaje de aprobación:

$$(1 / 1) \times 100 = 100\%$$

- Tiempo de respuesta observado: 949 ms
- Tiempo esperado: < 1000 ms
- Resultado: Aprobado

Observaciones

La ausencia de un campo de total en el carrito limita la validación completa de las reglas de negocio, lo cual representa una restricción significativa en pruebas de integración.

Pruebas negativas obligatorias

Se realizaron pruebas negativas con el objetivo de evaluar el comportamiento del API ante entradas inválidas.

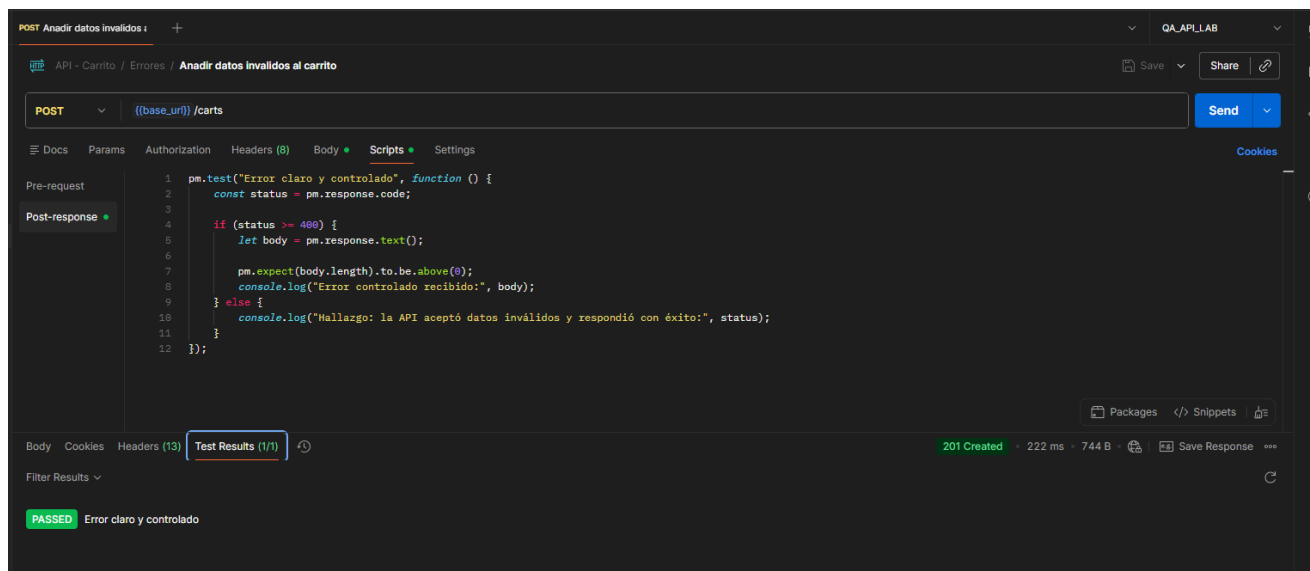
Enviar cantidad cero o negativa.

Se simuló el envío de un producto inválido mediante un identificador negativo (id = -1), con el fin de representar una condición incorrecta equivalente a una cantidad inválida o inexistente dentro del sistema.

```
{
  "userId": 1,
  "products": [
    {
      "id": -1
    }
  ]
}
```

Resultado esperado

Código 400 (Bad Request).



Validaciones realizadas

- Verificación de manejo de errores (esperado ≥ 400).
- Validación de existencia de mensaje de error en el body.
- Validación de tiempo de respuesta (< 1000 ms).
- Validación de estructura de respuesta.

Resultados obtenidos

- Código de respuesta: 201 (Created)
- Tiempo de respuesta: 495 ms
- Validaciones ejecutadas: 4
- Validaciones aprobadas: 4

Análisis del resultado

El API aceptó el request con datos inválidos y generó una respuesta exitosa, lo cual indica que no se realizan validaciones adecuadas sobre la validez de los productos enviados.

Este comportamiento representa una falla crítica en la validación de entrada, ya que permite la inserción de datos inconsistentes en el sistema.

Métricas asociadas

- Porcentaje de aprobación técnica:

$$(4 / 4) \times 100 = 100\%$$

- Resultado funcional: Fallido
- Severidad del defecto: Alta
- Impacto: Integridad de datos

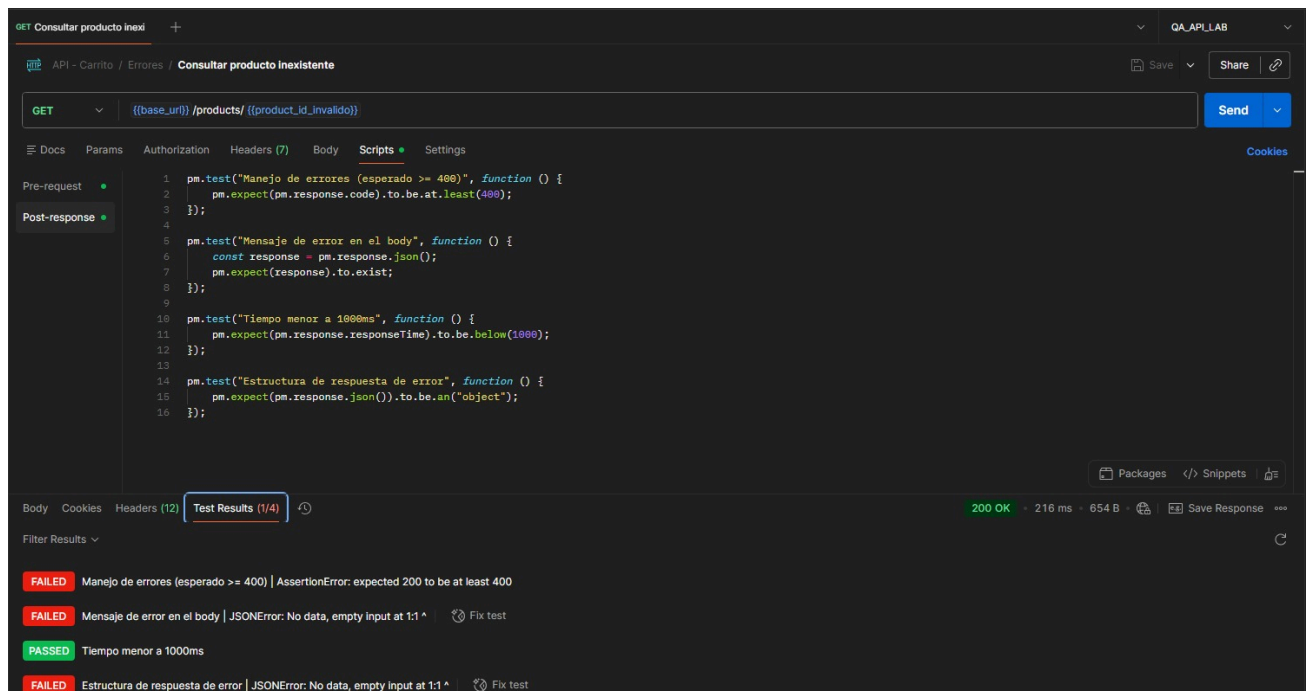
Consultar un producto inexistente

Se simuló la consulta de un producto inexistente mediante un identificador negativo (id = -10), con el fin de representar una condición incorrecta equivalente a un producto que no existe dentro del sistema.

GET /products/-10

Resultado esperado

Código 400 (Bad Request).



Validaciones realizadas

- Verificación de manejo de errores (esperado ≥ 400).
- Validación de existencia de mensaje de error en el body.
- Validación de tiempo de respuesta (< 1000 ms).
- Validación de estructura de respuesta de error.

Resultados obtenidos

- Código de respuesta: 200 OK
- Tiempo de respuesta: 215 ms
- Validaciones ejecutadas: 4
- Validaciones aprobadas: 1

Análisis del resultado

El API aceptó el request con datos inválidos y generó una respuesta exitosa, lo cual indica que no se realizan validaciones adecuadas sobre los productos enviados. Este comportamiento representa una falla crítica en la validación de entrada, ya que no devuelve mensajes de error descriptivos a pesar de buscar datos inexistentes.

Métricas asociadas

- Porcentaje de aprobación técnica:

$$(1 / 4) \times 100 = 25\%$$

- Resultado funcional: Fallido
- Severidad del defecto: Alta
- Impacto: Integridad de datos

Enviar tipo de dato incorrecto.

Se evaluó la validación de tipos de datos mediante el envío de valores incorrectos en el request, utilizando cadenas de texto en campos que deberían ser numéricos.

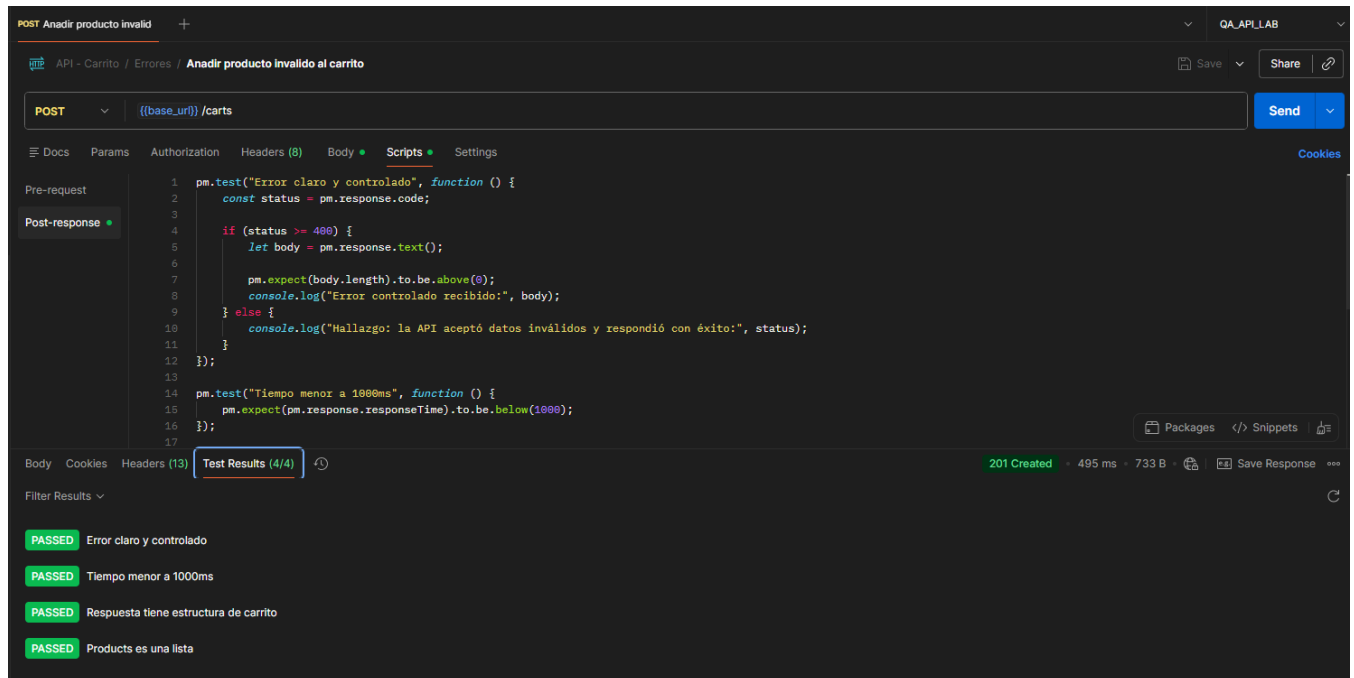
```
{
  "userId": "uno",
  "products": [
    {
      "id": "abc"
    }
  ]
}
```

Resultado esperado

Código 400 (Bad Request).

Resultado obtenido

La API aceptó los datos sin generar error.



Validaciones realizadas

- Verificación de código de error esperado (≥ 400).
- Validación de respuesta estructurada.
- Análisis del comportamiento del API ante datos inválidos.

Resultados obtenidos

- Código de respuesta: 201 (Created)
- Tiempo de respuesta: 222 ms
- Validaciones ejecutadas: 1
- Validaciones aprobadas: 1

Análisis del resultado

El API aceptó los datos con tipos incorrectos sin generar ningún error. Esto evidencia la ausencia de validaciones de contrato y control de tipos, lo cual compromete la confiabilidad del sistema.

Métricas asociadas

- **Porcentaje de aprobación técnica:**

$$(1 / 1) \times 100 = 100\%$$

- **Resultado funcional:** Fallido
- **Severidad del defecto:** Alta
- **Impacto:** Consistencia del sistema

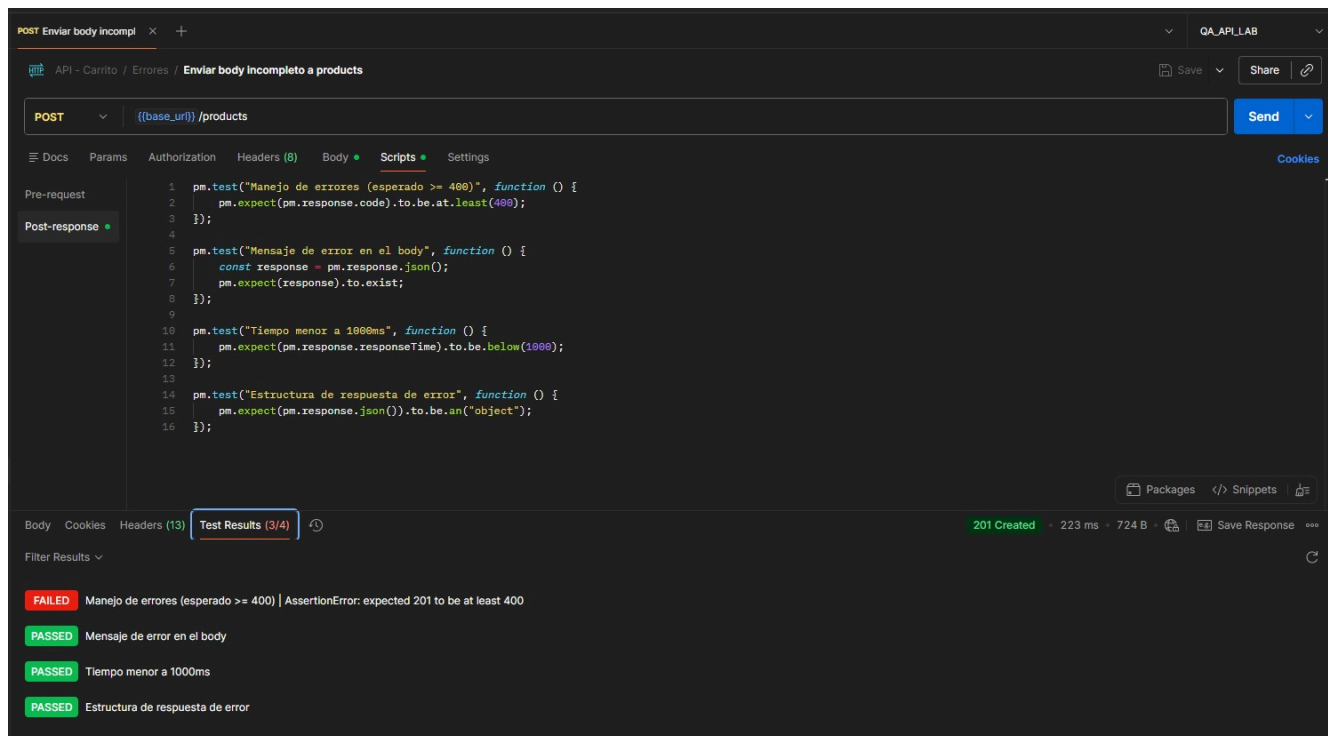
Enviar body incompleto a /products

Se simuló el envío de un producto con body incompleto, omitiendo campos obligatorios como price, description, category e image, con el fin de evaluar si el sistema valida la integridad de los datos antes de procesar la solicitud.

```
POST /products
{
  "id": 1,
  "title": "string"
}
```

Resultado esperado

Código 400 (Bad Request).



Validaciones realizadas

- Verificación de manejo de errores (esperado ≥ 400).

- Validación de existencia de mensaje de error en el body.
- Validación de tiempo de respuesta (< 1000 ms).
- Validación de estructura de respuesta de error.

Resultados obtenidos

- Código de respuesta: 200 OK
- Tiempo de respuesta: 223 ms
- Validaciones ejecutadas: 4
- Validaciones aprobadas: 3

Análisis del resultado

El API aceptó el request con un body incompleto y generó una respuesta exitosa, lo cual indica que no se realizan validaciones sobre los campos obligatorios del producto. El sistema procesó la solicitud sin exigir price, description, category ni image, devolviendo un objeto sin rechazar la entrada. A pesar de devolver un código de éxito, sí generó un mensaje de error claro en su estructura.

Métricas asociadas

- Porcentaje de aprobación técnica:
$$(3 / 4) \times 100 = 75\%$$
- Resultado funcional: Aprobado parcialmente
- Severidad del defecto: Alta
- Impacto: Integridad de datos

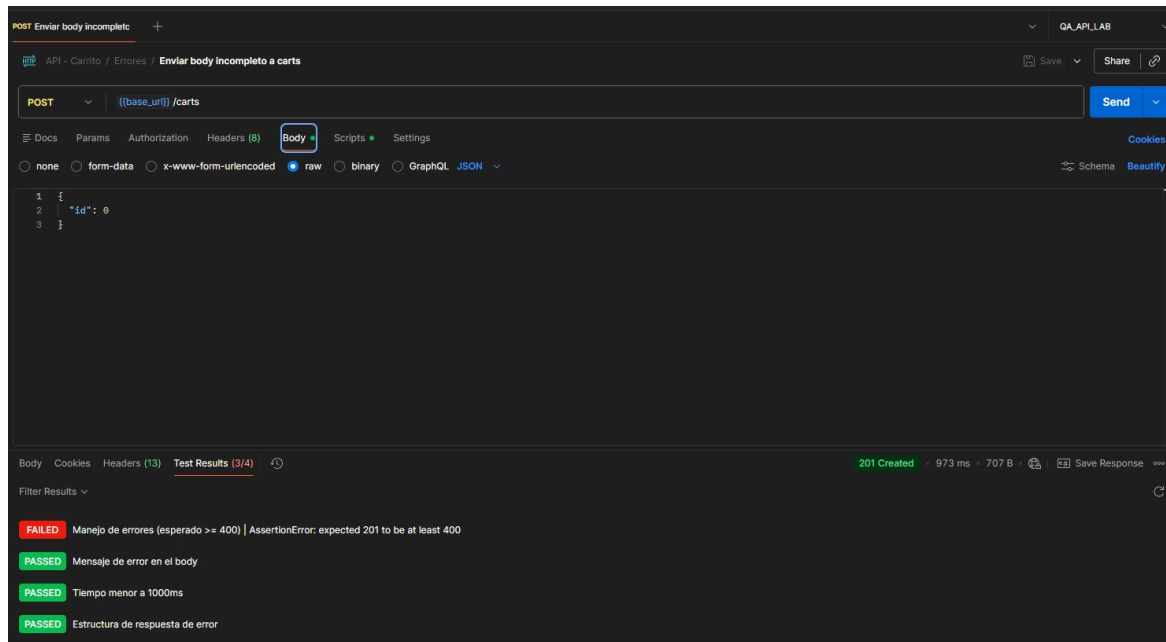
Enviar body incompleto a /carts

Se simuló el envío de un carrito con body incompleto, omitiendo campos obligatorios como userId y products, con el fin de evaluar si el sistema valida la integridad de los datos antes de procesar la solicitud.

```
POST /carts
{
  "id": 0
}
```

Resultado esperado

Código 400 (Bad Request).



Validaciones realizadas

- Verificación de manejo de errores (esperado ≥ 400).
- Validación de existencia de mensaje de error en el body.
- Validación de tiempo de respuesta (< 1000 ms).
- Validación de estructura de respuesta de error.

Resultados obtenidos

- Código de respuesta: 200 OK
- Tiempo de respuesta: 973 ms
- Validaciones ejecutadas: 4
- Validaciones aprobadas: 3

Análisis del resultado

El API aceptó el request con un body incompleto y generó una respuesta exitosa, lo cual indica que no se realizan validaciones sobre los campos obligatorios del carrito. El sistema procesó la solicitud sin exigir `userId` ni `products`, devolviendo un objeto sin rechazar la entrada. A pesar de esto, sí generó un mensaje de error.

Métricas asociadas

- Porcentaje de aprobación técnica:

$$(3 / 4) \times 100 = 75\%$$

- Resultado funcional: Aprobado parcialmente.
- Severidad del defecto: Alta
- Impacto: Integridad de datos

Validar errores claros y controlados.

Descripción de la prueba

Se evaluó si el sistema responde adecuadamente ante entradas inválidas mediante:

- Códigos de error apropiados (4xx).
- Mensajes descriptivos en la respuesta.

Resultados obtenidos

En los escenarios evaluados, el API respondió con códigos exitosos (201), aun cuando los datos enviados eran inválidos.

Adicionalmente:

- No se generaron mensajes de error.
- No se evidenció control de validaciones.
- El sistema procesa las solicitudes como válidas.

Análisis del resultado

El sistema no cumple con el principio de manejo de errores controlados, ya que:

- No detecta entradas inválidas.
- No retorna códigos HTTP adecuados.
- No proporciona retroalimentación clara al usuario.

Esto dificulta la detección de fallos y afecta la calidad del sistema.

Métricas asociadas

- **Tasa de error controlado:** 0%
- **Tasa de aceptación de datos inválidos:** 100%
- **Resultado global:** Fallido

Las pruebas negativas realizadas evidencian deficiencias críticas en la validación de entradas y manejo de errores del API.

A pesar de que todas las validaciones técnicas (tests en Postman) fueron aprobadas, el comportamiento funcional del sistema es incorrecto, ya que acepta datos inválidos sin generar errores.

Ejecución de colección completa

Se realizó la ejecución completa de la colección de pruebas utilizando la herramienta Postman Collection Runner, con el objetivo de validar el comportamiento de los distintos endpoints definidos para la API FakeStoreAPI.

La ejecución se llevó a cabo en el entorno QA_API_LAB, configurando un total de tres iteraciones, lo que permitió evaluar la consistencia de las respuestas del sistema ante ejecuciones repetidas.

De acuerdo con los resultados obtenidos , la colección ejecutó un total de pruebas sobre múltiples endpoints relacionados con productos y carritos.

Configuración de la ejecución

- Herramienta utilizada: Postman Collection Runner
- Entorno: QA_API_LAB
- Número de iteraciones: 3
- Total de pruebas ejecutadas: 117
- Duración total: aproximadamente 11 segundos
- Tiempo promedio de respuesta: 242 ms
- Total de errores de ejecución: 0

La ejecución múltiple permitió verificar la estabilidad del API y la repetibilidad de los resultados.

API - Carrito - Run results

Debug with AI

Run Again

New Run

Automate Run

Share

...

Ran today at 10:03:51 AM

View all runs

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	QA_API_LAB	3	11s 310ms	117	0	242 ms

RUN SUMMARY

View Results

			1	2	3
▶ GET	Consultar Productos	15 0			
▶ POST	Añadir Producto	12 0			
▶ GET	Consultar un producto por Id	6 6	✖		
▶ POST	Anadir carrito	12 0			
▶ GET	Obtener un carrito	6 6	✖		
▶ POST	Calcular Total Carrito	3 0			
▶ POST	Anadir datos Invalidos al carrito	3 0			
▶ POST	Anadir producto invalido al carrito	12 0			
▶ GET	Consultar producto Inexistente	3 9	✖		
▶ POST	Enviar body Incompleto a carts	9 3	✖		
▶ POST	Enviar body Incompleto a products	9 3	✖		

Resultados generales

Los resultados obtenidos muestran una combinación de pruebas exitosas y fallidas:

- Pruebas aprobadas: 90
- Pruebas fallidas: 27

Esto indica que el sistema presenta un buen comportamiento en pruebas funcionales básicas, pero evidencia inconsistencias en validaciones más estrictas.

Análisis de las pruebas realizadas

Las pruebas que resultaron exitosas corresponden principalmente a:

- Consulta de productos
- Creación de productos
- Creación de carritos
- Validación de estructura básica de respuestas
- Validación de tiempos de respuesta menores a 1000 ms
- Cálculo simulado del total del carrito

Estos resultados evidencian que la API responde correctamente en escenarios funcionales estándar, cumpliendo con los contratos básicos esperados.

Las pruebas fallidas se concentraron principalmente en los siguientes escenarios:

- Consulta de producto por ID
- Obtención de carrito
- Consulta de producto inexistente
- Envío de body incompleto
- Validación de manejo de errores

El análisis de estos resultados indica que:

- La API no valida correctamente la existencia de recursos.
- No se generan códigos de error adecuados (se esperaban códigos ≥ 400).
- Se aceptan datos inválidos sin validación estricta.
- El manejo de errores no es consistente ni controlado.

Hallazgos relevantes

Durante la ejecución se identificaron los siguientes hallazgos:

- La API responde con códigos exitosos (200/201) incluso ante datos inválidos.
- No existe validación robusta de integridad de datos.
- La estructura de algunas respuestas no coincide con lo esperado.

- El manejo de errores no sigue estándares REST adecuados.

Estos hallazgos evidencian limitaciones propias de una API de prueba como FakeStoreAPI.

La ejecución de la colección completa permitió evaluar de manera integral el comportamiento del API bajo diferentes escenarios y múltiples iteraciones.

Si bien el API presenta un desempeño adecuado en funcionalidades básicas, se identificaron debilidades significativas en el manejo de errores y validación de datos.

Las pruebas fallidas no corresponden a errores de implementación de las pruebas, sino a limitaciones propias del API utilizado, lo cual fue documentado como parte del análisis.

La ejecución mediante múltiples iteraciones permitió confirmar la estabilidad del sistema y la consistencia de los resultados, cumpliendo así con los objetivos del laboratorio de pruebas de integración.

Pruebas de integración realizadas con Pytest

Consultar API pública y validar contrato

Se realizó la consulta al endpoint público GET /products de FakeStoreAPI con el propósito de validar la disponibilidad del servicio y la estructura de los datos retornados.

Esta prueba corresponde a una validación de contrato, en la cual se verifica que la respuesta cumpla con los tipos de datos y atributos definidos.

Validaciones realizadas

- Código de estado HTTP igual a **200 (OK)**.
- La respuesta es de tipo **lista (array)**.
- La lista contiene al menos un elemento.

Se verificó que la respuesta tuviera código HTTP 200, que el contenido fuera una lista no vacía y que el primer producto incluyera los campos id, title, price, description, category e image. Además, se validó que el campo price correspondiera a un valor numérico.

Esta prueba permite comprobar el contrato básico del API antes de utilizar sus datos en otros procesos de integración.



```
5 #----- 2 Validar total del carrito -----
6 #Creacion de la primera prueba de integracion, calcular el total del carrito
7 def test_validar_total_carrito(db_connection):
8     cursor = db_connection.cursor()
9
10     # Obtener productos reales desde la API
11     response = requests.get(f'{BASE_URL}/products', timeout=10)
12     assert response.status_code == 200
13
14     products = response.json()[0:3]
15     cantidades = [2, 1, 3]
16     user_id = 1
17
18     productos_carrito = [
19         (
```

OBLEMS DEBUG CONSOLE TERMINAL

✓ TERMINAL

```
PS C:\Users\caray\OneDrive - Espy -m pytest qa_api_lab/tests/test_api_integration.py::test_validar_total_carrito -o
```

===== test session starts =====

platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0

rootdir: C:\Users\caray\OneDrive - Estudiantes ITCR\Documentos\TEC\S1-2026\Aseguramiento\TAREAS\LAB 9\GIT\Lab_Integracion

plugins: anyio-4.10.0

collected 1 item

qa_api_lab\tests\test_api_integration.py . [100%]

1 passed in 2.17s

PS C:\Users\caray\OneDrive - Estudiantes ITCR\Documentos\TEC\S1-2026\Aseguramiento\TAREAS\LAB 9\GIT\Lab_Integracion>

Insertar producto consultado en SQLite

Se realizó la consulta al endpoint público GET /products de FakeStoreAPI y se extrajo el primer producto con el fin de comprobar la inserción exitosa de un producto en la base de datos.

Validaciones realizadas

- Código de estado HTTP igual a 200 (OK).
- Inserción exitosa del producto en la base de datos
- Total de registros en el carrito igual a 1

Resultados obtenidos

El API respondió correctamente y el producto se guardó en la base de datos de manera exitosa. Sin embargo, la duración fue de 1.81 segundos superando el umbral de 1 segundo.

```
PS C:\Users\HP\OneDrive\Documentos\GitHub\Lab_Integracion> pytest .\qa_api_lab\tests\test_api_integration.py::test_insertar_producto
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-8.4.1, pluggy-1.6.0
rootdir: C:\Users\HP\OneDrive\Documentos\GitHub\Lab_Integracion
plugins: anyio-4.12.0
collected 1 item

qa_api_lab\tests\test_api_integration.py . [100%]

----- 1 passed in 1.81s -----
PS C:\Users\HP\OneDrive\Documentos\GitHub\Lab_Integracion>
```

Validaciones: $(3 / 3) \times 100 = 100\%$

Resultado: Aprobado

Validar total del carrito

La presente prueba de integración tuvo como objetivo validar el cálculo del total del carrito mediante la combinación de datos obtenidos desde una API pública y su posterior almacenamiento en una base de datos SQLite en memoria.

Se buscó verificar que el sistema fuera capaz de calcular correctamente tanto el número total de productos como el monto total acumulado, aplicando las reglas de negocio correspondiente.

Flujo de integración

La prueba se ejecutó siguiendo el siguiente flujo:

1. Consulta de productos desde la API (GET /products)

2. Selección de múltiples productos
3. Asignación de cantidades simuladas
4. Inserción de datos en SQLite
5. Cálculo del total esperado
6. Consulta de totales en la base de datos
7. Comparación de resultados

Datos utilizados

Para la ejecución de la prueba se consideraron los siguientes elementos:

- Tres productos obtenidos desde la API pública.
- Cantidades simuladas asignadas a cada producto:

Producto 1 → cantidad = 2

Producto 2 → cantidad = 1

Producto 3 → cantidad = 3

- Identificador de usuario utilizado para agrupar los registros: `user_id = 1`.

```
PS C:\Users\caray\OneDrive - Estudiantes ITCR\Documentos\TEC\S1-2026\Aseguramiento\TAREAS\LAB 9\GIT\Lab_Integracion> py -m pytest qa_a
pi_lab/tests/test_api_integration.py::test_validar_total_carrito
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\caray\OneDrive - Estudiantes ITCR\Documentos\TEC\S1-2026\Aseguramiento\TAREAS\LAB 9\GIT\Lab_Integracion
plugins: anyio-4.10.0
collected 1 item

qa_api_lab\tests\test_api_integration.py . [100%]

===== 1 passed in 1.52s =====
PS C:\Users\caray\OneDrive - Estudiantes ITCR\Documentos\TEC\S1-2026\Aseguramiento\TAREAS\LAB 9\GIT\Lab_Integracion> |
```

Validaciones realizadas

Durante la ejecución se verificaron los siguientes aspectos:

- Respuesta exitosa del API (código HTTP 200).
- Inserción correcta de los registros en la base de datos SQLite.
- Cálculo correcto del total esperado de productos.
- Cálculo correcto del monto total esperado.
- Comparación entre los valores esperados y los obtenidos desde la base de datos.

- Validación de precisión en valores decimales mediante `pytest . approx`.

Los resultados obtenidos fueron consistentes con los valores esperados:

- El total de productos almacenados coincide con la suma de las cantidades definidas.
- El monto total calculado coincide con el valor esperado.
- No se presentaron inconsistencias en la persistencia ni en los cálculos realizados.

Métricas asociadas

- **Validaciones ejecutadas:** 5
- **Validaciones aprobadas:** 5
- **Porcentaje de aprobación:**

$$(5 / 5) \times 100 = 100\%$$

- **Cobertura de regla de negocio:** 100%
- **Tipo de prueba:** Integración + persistencia + lógica de negocio

Observaciones

- La API FakeStoreAPI no proporciona información sobre cantidades ni el cálculo del total del carrito.
- Fue necesario simular la cantidad dentro del entorno de prueba.
- El cálculo del total se realizó mediante lógica adicional en Pytest.
- La prueba demuestra integración entre datos externos (API) y persistencia local (SQLite).

Validar cantidades

Se realizó una prueba con el objetivo de validar que el sistema no permita el registro de cantidades inválidas dentro del carrito, específicamente valores negativos o iguales a cero.

Esta validación forma parte de las reglas de negocio del sistema, orientadas a garantizar la consistencia e integridad de los datos asociados al carrito de compras.

Es importante destacar que la API pública utilizada (FakeStoreAPI) no maneja el concepto de cantidades dentro del endpoint evaluado.

Debido a esta limitación, la validación de cantidades no puede realizarse directamente sobre el API, por lo que fue necesario implementar esta lógica a nivel de persistencia y pruebas automatizadas utilizando SQLite en memoria.

Regla de negocio evaluada

La cantidad de productos debe cumplir la siguiente condición:

cantidad > 0

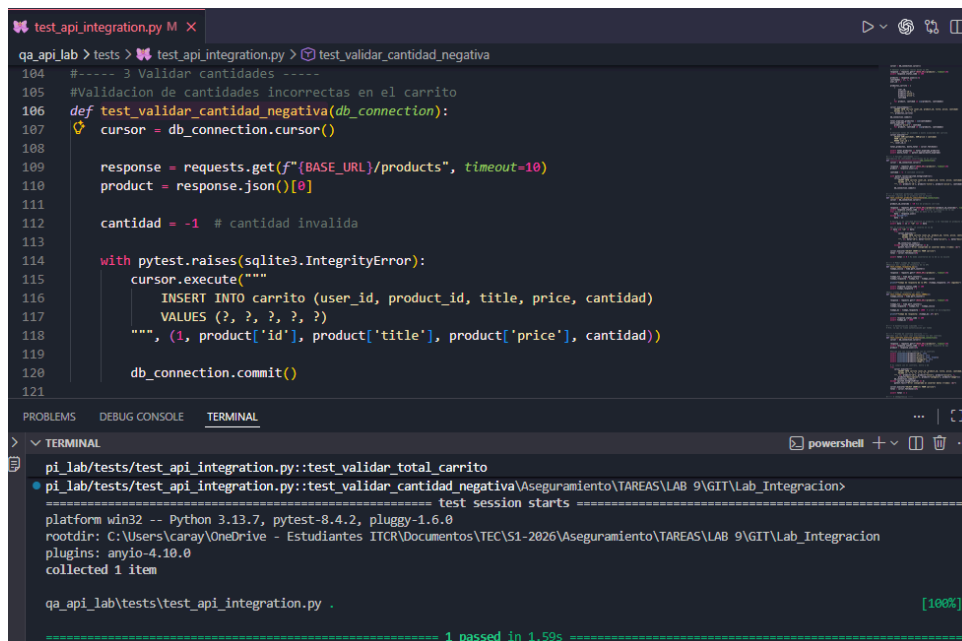
Se implementó una restricción en la base de datos SQLite mediante la cláusula CHECK, la cual impide la inserción de valores menores o iguales a cero en el campo cantidad.

Validaciones realizadas

- Consulta de producto válido desde la API.
- Intento de inserción con cantidad negativa.
- Verificación de generación de error (IntegrityError).

Resultados obtenidos

La base de datos rechazó correctamente la inserción de la cantidad negativa, generando un error controlado. Esto confirma que la restricción definida funciona adecuadamente.



```
test_api_integration.py M X
qa_api_lab > tests > test_api_integration.py > test_validar_cantidad_negativa
104 #---- 3 Validar cantidades ----
105 #Validacion de cantidades incorrectas en el carrito
106 def test_validar_cantidad_negativa(db_connection):
107     cursor = db_connection.cursor()
108
109     response = requests.get(f"{BASE_URL}/products", timeout=10)
110     product = response.json()[0]
111
112     cantidad = -1 # cantidad invalida
113
114     with pytest.raises(sqlite3.IntegrityError):
115         cursor.execute("""
116             INSERT INTO carrito (user_id, product_id, title, price, cantidad)
117             VALUES (?, ?, ?, ?, ?)
118             """, (1, product['id'], product['title'], product['price'], cantidad))
119
120     db_connection.commit()
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2
```

- **Cobertura de regla de negocio:** 100%
- **Tipo de validación:** Integridad de datos + regla de negocio

Observaciones

- FakeStoreAPI no contempla el manejo de cantidades en el endpoint utilizado.
- Fue necesario implementar la validación como una regla de negocio local.
- La restricción en SQLite permitió simular el comportamiento esperado de un sistema productivo.
- Esta prueba complementa la validación del sistema más allá de las capacidades del API.

Ingresar productos inexistentes

Esta prueba evaluó que el sistema no registre en la base de datos un producto inexistente en la API.

Validaciones realizadas

- Código de estado HTTP igual a 200
- La respuesta de la API para el id inválido retorna vacío.
- No se realiza ninguna inserción en la base de datos
- Total de registros en el carrito igual a 0

Resultados obtenidos

A pesar de que la API tuvo una respuesta exitosa de 200 OK, retornó un conjunto vacío para el id inválido y el sistema no insertó ningún registro.

```
119
120 #----- 4 Ingresar productos inexistentes -----
121 # Validar ingreso de un producto que no existe
122 def test_insertar_producto_inexistente(db_connection):
123     cursor = db_connection.cursor()
124
125     product_id_invalido = -10 #id de producto inválido
126
127     response = requests.get(f"{BASE_URL}/products/{product_id_invalido}", timeout=10)
128     assert response.status_code == 200 #valida la respuesta de la api
129     try: #puede que no venga nada en data si es inválido
130         data = response.json()
131     except ValueError:
132         data = {}
133
134     # Valida si en realidad devolvió un producto, o en realidad el producto no existe
135     assert data == {} or "id" not in data
136
137     #Si el producto existe, se inserta en la bd
138     if data and "id" in data:
139         try:
140             cursor.execute("""
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
2688
2689
2690
2691
2692
2693
2694
2695
2696
2697
2698
2699
2700
2701
2702
2703
2704
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715
2716
2717
2718
2719
2720
2721
2722
2723
2724
2725
2726
2727
2728

```

```
70 #Medir tiempo de respuesta (< 1000 ms)
71 def test_tiempo_respuesta_api_menos_1000ms():
72     tiempo_inicio = time.perf_counter()
73
74     response = requests.get(f'{BASE_URL}/products', timeout=10)
75
76     tiempo_fin = time.perf_counter()
77     tiempo_respuesta = tiempo_fin - tiempo_inicio
78
79     tiempo_ms = tiempo_respuesta * 1000 # probar en milisegundos
80
81     print(f"Tiempo de respuesta: {tiempo_ms:.2f} ms")
82
83     assert response.status_code == 200
84     assert tiempo_ms < 1000
85
86
87 #
88
89 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\HP\OneDrive\Documentos\GitHub\Lab_Integracion> pytest .\qa_api_lab\tests\test_api_integration.py::test_tiempo_respuesta_api_menos_1000ms
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-8.4.1, pluggy-1.6.0
rootdir: C:\Users\HP\OneDrive\Documentos\GitHub\Lab_Integracion
plugins: anyio-4.12.0
collected 1 item

qa_api_lab\tests\test_api_integration.py .
===== 1 passed in 0.65s =====
PS C:\Users\HP\OneDrive\Documentos\GitHub\Lab_Integracion>
```

Métricas

Validaciones: $(2 / 2) \times 100 = 100\%$

Tiempo de respuesta observado: 650 ms

Tiempo esperado: < 1000 ms

Resultado: Aprobado

Prueba de contrato estricta

Esta prueba evaluó que los datos retornados por la API cumplan con los tipos de datos exactos definidos en el contrato.

Validaciones realizadas

- id es de tipo int
- title es de tipo str
- price es de tipo int o float
- category es de tipo str
- image es de tipo str
- Inserción exitosa en la base de datos sin errores de integridad

- Total de registros en el carrito igual a 1

Resultados obtenidos

El producto cumplió con todos los tipos definidos en el contrato y fue insertado correctamente en la base de datos.

```

92 #Validar que los tipos sean exactamente los del contrato
93 def test_contrato_estriicto_productos(db_connection):
94     cursor = db_connection.cursor()
95
96     response = requests.get(f"{BASE_URL}/products", timeout=10)
97     assert response.status_code == 200 #valida respuesta de api
98     product = response.json()[0]
99
100     #Valida que los tipos cumplan con el contrato
101     assert isinstance(product["id"], int)
102     assert isinstance(product["title"], str)
103     assert isinstance(product["price"], (int, float))
104     assert isinstance(product["category"], str)
105     assert isinstance(product["image"], str)
106
107     # Si cumple con el contrato, entra a bd
108     try:
109         cursor.execute("""
110             INSERT INTO carrito (user_id, product_id, title, price, cantidad, description, category, image)
111             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
112             """, (1, product["id"], product["title"], product["price"], 1,
113                 product["description"], product["category"], product["image"]))
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Métricas

Validaciones: $(7 / 7) \times 100 = 100\%$

Resultado: Aprobado

Idempotencia

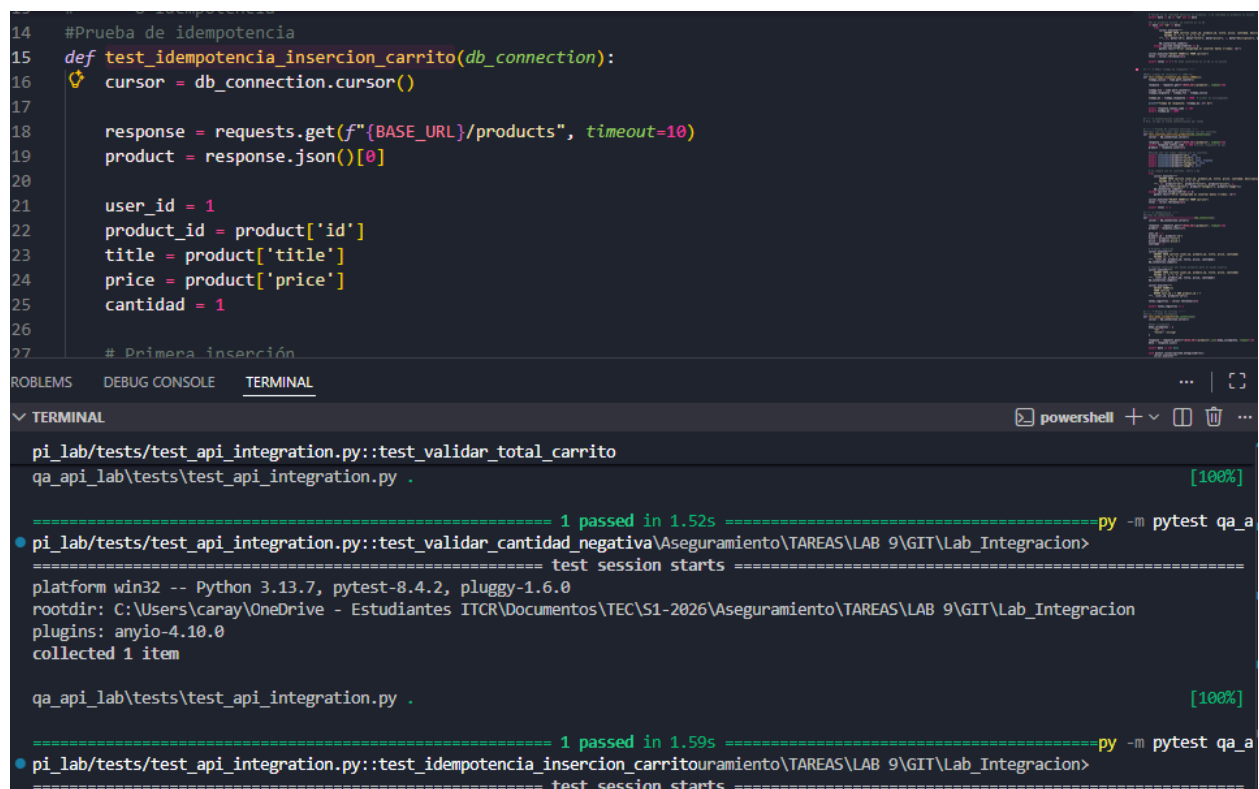
Se realizó una prueba de idempotencia con el objetivo de evaluar el comportamiento del sistema al ejecutar dos veces la misma operación de inserción de un producto en el carrito.

La idempotencia es una propiedad que determina si una operación puede repetirse múltiples veces sin alterar el resultado final del sistema. En un contexto ideal, una operación idempotente debería mantener la consistencia de los datos sin generar duplicados.

La API pública utilizada (FakeStoreAPI) no implementa lógica de carrito persistente ni control de duplicados. Por esta razón, la validación de idempotencia se realizó a nivel de persistencia local utilizando SQLite en memoria.

La prueba se ejecutó siguiendo el siguiente flujo:

1. Consulta de un producto desde la API (GET /products)
2. Extracción de los datos del producto
3. Inserción del producto en SQLite
4. Repetición de la misma inserción
5. Consulta de registros en la base de datos
6. Verificación de duplicados



```
14 #Prueba de idempotencia
15 def test_idempotencia_insercion_carrito(db_connection):
16     cursor = db_connection.cursor()
17
18     response = requests.get(f"{BASE_URL}/products", timeout=10)
19     product = response.json()[0]
20
21     user_id = 1
22     product_id = product['id']
23     title = product['title']
24     price = product['price']
25     cantidad = 1
26
27     # Primera inserción
```

pi_lab/tests/test_api_integration.py::test_validar_total_carrito
qa_api_lab\tests\test_api_integration.py . [100%]

===== 1 passed in 1.52s =====
py -m pytest qa_a
pi_lab/tests/test_api_integration.py::test_validar_cantidad_negativa\Aseguramiento\TAREAS\LAB 9\GIT\Lab_Integracion>
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\caray\OneDrive - Estudiantes ITCR\Documentos\TEC\S1-2026\Aseguramiento\TAREAS\LAB 9\GIT\Lab_Integracion
plugins: anyio-4.10.0
collected 1 item
qa_api_lab\tests\test_api_integration.py . [100%]

===== 1 passed in 1.59s =====
py -m pytest qa_a
pi_lab/tests/test_api_integration.py::test_idempotencia_insercion_carritouramiento\TAREAS\LAB 9\GIT\Lab_Integracion>
===== test session starts =====

Validaciones realizadas

- Verificación de respuesta exitosa del API.
- Inserción del producto en la base de datos.
- Repetición de la misma operación con los mismos datos.
- Conteo de registros para el mismo user_id y product_id.
- Validación del número total de registros generados.

Resultados obtenidos

El resultado de la consulta fue:

total_registros = 2

Esto indica que ambas inserciones fueron ejecutadas sin restricción, generando dos registros idénticos en el carrito.

El comportamiento observado evidencia que la operación no es idempotente, ya que cada ejecución genera un nuevo registro en lugar de mantener el estado del sistema.

Métricas asociadas

- **Operaciones ejecutadas:** 2
- **Registros generados:** 2
- **Duplicados detectados:** 1
- **Resultado:** No idempotente
- **Severidad:** Media
- **Consistencia del sistema:** Baja

FakeStoreAPI no gestiona carritos reales ni lógica de control de duplicados. La validación se realizó mediante una base de datos SQLite en memoria. No se implementaron restricciones que impidan la inserción repetida del mismo producto. El comportamiento observado es consistente con una operación de inserción simple sin control de integridad.

Manejo de errores

Esta prueba evaluó el manejo de errores de la api y el sistema al forzar la inserción de un body incompleto.

Validaciones realizadas

- La API acepta el body incompleto y retorna una respuesta no nula.
- El intento de inserción en la base de datos lanza un error de integración.
- Total de registros en el carrito igual a 0.

Resultados obtenidos

El producto cumplió con todas las validaciones y lanzó el error correspondiente al ser insertada en la base de datos..

```
263 def test_body_incompleto(db_connection):
264     cursor = db_connection.cursor()
265
266     #body incompleto
267     body_incompleto = {
268         "id": 1,
269         "title": "string"
270     }
271
272     response = requests.post(f"{BASE_URL}/products", json=body_incompleto, timeout=10)
273     data = response.json()
274
275     assert data is not None
276
277     with pytest.raises(sqlite3.IntegrityError):
278         cursor.execute("""
279             INSERT INTO carrito (user_id, product_id, title, price, cantidad, description, category, image)
280             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
281             """, (1, data.get("id"), data.get("title"), data.get("price"), 1,
282                 data.get("description"), data.get("category"), data.get("image")))
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

```
PS C:\Users\HP\OneDrive\Documentos\GitHub\Lab_Integracion> pytest .\qa_api_lab\tests\test_api_integration.py::test_body_incompleto
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-8.4.1, pluggy-1.6.0
rootdir: C:\Users\HP\OneDrive\Documentos\GitHub\Lab_Integracion
plugins: anyio-4.12.0
collected 1 item

qa_api_lab\tests\test_api_integration.py .

===== 1 passed in 1.14s =====
PS C:\Users\HP\OneDrive\Documentos\GitHub\Lab_Integracion>
```

Métricas

Validaciones: $(3 / 3) \times 100 = 100\%$

Resultado: Aprobado

Prueba de integración parcial

La prueba de integración parcial tuvo como objetivo simular un escenario en el cual la API responde correctamente, pero ocurre un fallo durante el proceso de almacenamiento de datos en la base de datos SQLite en memoria.

Este tipo de prueba permite validar el comportamiento del sistema ante errores internos que pueden presentarse después de recibir una respuesta exitosa desde un servicio externo.

La prueba se ejecutó mediante el siguiente flujo:

1. Consulta de producto desde la API pública
2. Validación de respuesta HTTP correcta
3. Intento de inserción en SQLite
4. Generación de fallo en la base de datos

5. Captura y validación del error

```
===== 1 passed in 1.02s ===== py -m pytest qa_a
• pi_lab/tests/test_api_integration.py::test_integracion_parcial_api_ok_db_fallaento[TAREAS\LAB 9\GIT\Lab_Integracion]
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\caray\OneDrive - Estudiantes ITCR\Documentos\TEC\S1-2026\Aseguramiento\TAREAS\LAB 9\GIT\Lab_Integracion
plugins: anyio-4.10.0
collected 1 item

qa_api_lab\tests\test_api_integration.py . [100%]

===== 1 passed in 0.90s =====
```

Se realizó una solicitud al endpoint:

GET /products

Posteriormente, se intentó almacenar el producto en SQLite omitiendo intencionalmente un campo obligatorio de la tabla. Debido a que la estructura de la base de datos define restricciones NOT NULL, SQLite generó un error de integridad (IntegrityError).

Esta estrategia permitió simular un fallo real de persistencia sin afectar la respuesta exitosa del API.

Validaciones realizadas

Durante la prueba se verificaron los siguientes aspectos:

- Respuesta HTTP correcta del API (200 OK).
- Obtención válida de datos del producto.
- Generación controlada del error en SQLite.
- Correcta captura del error mediante Pytest.
- Confirmación de que el fallo ocurre en la persistencia y no en el consumo del API.

La API respondió correctamente y retornó un producto válido. Sin embargo, al intentar almacenar la información en SQLite, la base de datos rechazó la operación debido al incumplimiento de restricciones definidas en la tabla.

El sistema generó un error controlado (IntegrityError), confirmando que el fallo fue detectado correctamente durante el proceso de integración.

Mock opcional

Esta prueba evaluó la respuesta del sistema cuando la API está caída.

Validaciones realizadas

- La llamada a requests.get lanza una excepción simulada.
- El sistema captura correctamente la excepción

- Total de registros en el carrito igual a 0 (no hubo inserción)

Resultados obtenidos

El producto cumplió con lo esperado y al capturar la excepción detuvo la transacción y no insertó el producto al carrito.

```

322 #----- 11 Mock opcional -----
323 def test_mock_api_caida(db_connection, monkeypatch):
324     cursor = db_connection.cursor()
325
326     fake_requests = MagicMock() #se crea un requests falso y se simula que la API falla
327     fake_requests.get.side_effect = Exception("API falla")
328
329     monkeypatch.setitem(sys.modules, "requests", fake_requests) #cuando se intente llamar requests, se llamará al falso
330
331     import requests #se importa el falso
332
333     with pytest.raises(Exception):
334         requests.get("https://fakestoreapi.com/products") #se simula la falla
335
336     cursor.execute("SELECT COUNT(*) FROM carrito")
337     total = cursor.fetchone()[0]
338
339     assert total == 0
340
341

```

```

PROJ Focus folder in explorer (ctrl + click) TERMINAL PORTS powershell + - 
PS C:\Users\HP\OneDrive\Documentos\GitHub\Lab_Integracion> pytest .\qa_api_lab\tests\test_api_integration.py::test_mock_api_caida
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-8.4.1, pluggy-1.6.0
rootdir: C:\Users\HP\OneDrive\Documentos\GitHub\Lab_Integracion
plugins: anyio-4.12.0
collected 1 item

qa_api_lab\tests\test_api_integration.py .

===== 1 passed in 0.40s =====
PS C:\Users\HP\OneDrive\Documentos\GitHub\Lab_Integracion>

```

Métricas

Validaciones: $(3 / 3) \times 100 = 100\%$

Resultado: Aprobado

Comparación Postman y Pytest

En el desarrollo del presente laboratorio se utilizaron dos herramientas principales para la ejecución de pruebas sobre APIs: Postman y Pytest. Ambas permiten validar el comportamiento de servicios web, sin embargo, presentan diferencias significativas en cuanto a su enfoque, capacidades y casos de uso.

Postman se orienta principalmente a la ejecución manual y semi-automatizada de pruebas, mientras que Pytest permite la automatización completa mediante código, facilitando la integración con otros componentes del sistema.

Enfoque de cada herramienta

Postman es una herramienta gráfica que permite construir, enviar y validar solicitudes HTTP de manera interactiva. Su principal fortaleza radica en la facilidad de uso y en la visualización inmediata de resultados, lo que la hace ideal para pruebas exploratorias y validaciones rápidas.

Por otro lado, Pytest es un framework de pruebas automatizadas en Python que permite definir casos de prueba mediante código. Esto facilita la creación de pruebas más complejas, integrando lógica adicional, persistencia de datos y validaciones avanzadas.

Ambas herramientas son complementarias dentro del proceso de pruebas de software. Postman resulta ideal para etapas iniciales y validaciones rápidas, mientras que Pytest permite profundizar en la automatización y en la validación de escenarios complejos.

En el contexto del laboratorio, el uso combinado de ambas herramientas permitió obtener una visión integral del comportamiento del sistema, aprovechando la facilidad de uso de Postman y la capacidad de automatización y análisis de Pytest.

Manejo de errores y validaciones negativas

Una diferencia significativa fue la capacidad de cada herramienta de manejar errores y escenarios negativos en conjunto con validaciones de integridad. En Postman, las pruebas negativas dependen completamente del comportamiento de la API, y tiene más limitaciones para validar casos cuando esta devolvía datos erróneos pero con códigos de éxito como 200 o 201.

En cambio, Pytest permite integrar lógica adicional, por lo que no depende exclusivamente del comportamiento de la API. Puede implementar restricciones directamente en la base de datos, validar los datos devueltos, encontrar errores de integridad e incluso responder de manera correcta cuando la API deja de funcionar o está caída. De esta manera se pueden validar escenarios que no son manejados en la API. Esto comprueba que, a pesar de que Postman es una buena herramienta inicial para validar el comportamiento de la API con casos de prueba, herramientas como Pytest son necesarias para validar la lógica completa y manejar casos de error que una API no toma en cuenta.

Reporte Final

Prueba	Métricas (¿qué se probó?)	Resultado esperado	Hallazgos (resultado real)	Defectos (¿qué falló?)	Importancia (¿Por qué?)	Recomendación de mejora
Postman						
TC-1 Consultar catálogo	- Código de estado HTTP igual a 200. - Tiempo de respuesta inferior a 1000 ms. - Tipo de dato de la respuesta: arreglo. - Existencia de al menos un producto. - Estructura del objeto producto.	200 OK, lista no vacía, campos: id, title, price, description, category, image, respuesta menor a 1000 ms.	80% aprobado (4/5). Tiempo: 1018 ms, superó el umbral de 1000 ms	Tiempo de respuesta excede el límite definido (1018 ms > 1000 ms)	Media. El usuario puede consultar el catálogo completo. Un tiempo de respuesta ligeramente elevado no causará graves inconvenientes.	Se puede extender el tiempo máximo a 1500 ms para endpoints con una gran cantidad de objetos.
TC-2 Añadir un producto nuevo	- Código de estado HTTP igual a 201. - Tiempo de respuesta inferior a 1000 ms. - Existencia del producto creado en la respuesta. - Price es un número. - Estructura del objeto producto.	201 Created, objeto producto con id, title, price, description, category, image, respuesta menor a 1000 ms.	100% aprobado (4/4). Tiempo: 918 ms. Producto creado correctamente	Ninguno	Alta. El producto debe ser añadido siguiendo todos los estándares y reglas de negocio, de lo contrario habrá graves problemas de consistencia.	Se inserta exitosamente la información completa del producto.
TC-3 Consultar un producto	- Código de estado HTTP igual a 200 (OK). - Tiempo de respuesta inferior a 1000 milisegundos. - El producto retornado	200 OK, producto con id, title, price, description, category, image, respuesta menor a	Falló funcionalmente. (2/4). Tiempo: 212 ms. Producto sin existencia real,	El producto retornado no cumple con todos los campos	Alta. Al consultar un producto, este debe estar en existencia y retornar los datos	Agregar validación de existencia del producto.

Prueba	Métricas (¿qué se probó?)	Resultado esperado	Hallazgos (resultado real)	Defectos (¿qué falló?)	Importancia (¿Por qué?)	Recomendación de mejora
	existe en el catálogo. - Estructura del objeto producto.	1000 ms.	campos faltantes	del contrato	válidos para mantener confiabilidad.	
TC-4 Añadir nuevo carrito	- Código de estado HTTP igual a 201 (Created). - Presencia de los atributos. - Validación lista no vacía. - Confirmación de producto con identificador válido.	201 Created, carrito con id, userId y lista de products, respuesta menor a 1000 ms.	100% aprobado (4/4). Tiempo: 788 ms. Carrito creado exitosamente	Ninguno	Alta. Es importante que se cree el carrito de manera correcta para cada usuario con los productos existentes.	Se incluyen exitosamente los datos completos del carrito al ser creado.
TC-5 Obtener un carrito	- Código de estado HTTP igual a 200 (OK). - Tiempo de respuesta inferior a 1000 milisegundos. - El carrito retornado existe en el sistema. - Estructura del objeto carrito	200 OK, carrito existente con campos completos, respuesta menor a 1000 ms.	100% aprobado (4/4). Tiempo: 338 ms. Carrito retornado correctamente	Ninguno	Media. Usualmente no se va a consultar el objeto carrito como tal.	Se incluye exitosamente información detallada del carrito.
TC-6 Consultar y validar total del carrito	- El precio del producto es un valor válido (> 0). - El total calculado es correcto y mayor a cero.	Total calculado correctamente a partir del precio y la cantidad, respuesta menor a 1000 ms.	100% aprobado (1/1). Tiempo: 949 ms. Cálculo manual validado	Ninguno	Alta. Es importante conocer el valor exacto del total del carrito, ya que esto representa lo que el usuario comprará.	Agregar campo 'total' en el endpoint del carrito.
TC-7 Enviar cantidad inválida (simulada)	- Verificación de manejo de errores (esperado ≥ 400). - Validación de existencia de mensaje de error en el body.	400 Bad Request, mensaje de error descriptivo	Falló funcionalmente. API retornó 201 aceptando datos	La API no valida la existencia ni validez de	Alta. Es importante que la API no permita el ingreso de cantidades	Agregar validación de productos antes de crear el carrito.

Prueba	Métricas (¿qué se probó?)	Resultado esperado	Hallazgos (resultado real)	Defectos (¿qué falló?)	Importancia (¿Por qué?)	Recomendación de mejora
	- Validación de tiempo de respuesta (< 1000 ms). - Validación de estructura de respuesta.		inválidos	productos en el carrito	inválidas para mantener la consistencia.	
TC-8 Consultar un producto inexistente	- Verificación de manejo de errores (esperado ≥ 400). - Validación de existencia de mensaje de error en el body. - Validación de tiempo de respuesta (< 1000 ms). - Validación de estructura de respuesta de error.	400 Bad Request, respuesta menor a 1000 ms.	Falló funcionalmente. (1/4). API retornó 200 OK con respuesta vacía	API no maneja correctamente productos inexistentes, no retorna código de error	Alta. Es importante que la API devuelva un mensaje de error oportuno al intentar consultar un producto inexistente.	Agregar respuesta de error para productos no encontrados.
TC-9 Enviar tipo de dato incorrecto	- Verificación de código de error esperado (≥ 400). - Validación de respuesta estructurada. - Análisis del comportamiento del API ante datos inválidos. - Tiempo menor a 1000 ms.	400 Bad Request por tipo incorrecto, respuesta menor a 1000 ms.	Falló funcionalmente. API aceptó tipos incorrectos y retornó 201	No existe validación de tipos de datos en el API	Alta. Es importante que la API cumpla con las reglas de negocio establecidas y que solo acepte tipos de dato correctos.	Agregar validación de tipos.
TC-10 Enviar un body incompleto a /products.	- Verificación de manejo de errores (esperado ≥ 400). - Validación de existencia de mensaje de error en el body. - Validación de tiempo de respuesta (< 1000 ms). - Validación de estructura de	400 Bad Request, campos obligatorios requeridos, respuesta menor a 1000 ms.	75% aprobado (3/4). API aceptó body incompleto con 200 OK	No se validan campos obligatorios del producto	Alta. Es importante que el producto tenga toda la información completa correspondiente para evitar	Agregar validación de campos requeridos.

Prueba	Métricas (¿qué se probó?)	Resultado esperado	Hallazgos (resultado real)	Defectos (¿qué falló?)	Importancia (¿Por qué?)	Recomendación de mejora
	respuesta de error.				inconsistencias.	
TC-11 Enviar un body incompleto a /carts.	- Verificación de manejo de errores (esperado ≥ 400). - Validación de existencia de mensaje de error en el body. - Validación de tiempo de respuesta (< 1000 ms). - Validación de estructura de respuesta de error.	400 Bad Request, userId y products requeridos, respuesta menor a 1000 ms.	75% aprobado (3/4). API aceptó body incompleto con 200 OK	No se validan campos obligatorios del carrito	Alta. Es importante que el carrito tenga toda la información completa correspondiente para evitar inconsistencias.	Agregar validación de campos requeridos.
TC-12 Prueba de integración parcial	-Verificación de respuesta HTTP de la API (status_code == 200). -Validación de estructura del producto (id, title, price). -Verificación de manejo de errores en SQLite mediante excepción sqlite3.IntegrityError. -Validación de tolerancia a fallos durante la persistencia de datos.	La API debe responder correctamente al endpoint /products con código HTTP 200 y devolver un producto válido. Posteriormente, al intentar insertar un registro incompleto en SQLite (sin el campo title obligatorio), la base de datos debe generar un error controlado IntegrityError.	La API respondió exitosamente con estado 200 y devolvió un producto válido con los campos requeridos. Durante el proceso de inserción en la tabla carrito, SQLite detectó la ausencia del campo obligatorio title y lanzó correctamente la excepción sqlite3.IntegrityError.	No se detectaron defectos críticos en el manejo de errores. El sistema logró identificar y controlar adecuadamente la violación de integridad en la base de datos.	Alta. Esta prueba permite validar la resiliencia del sistema frente a fallos internos de persistencia, garantizando que los errores de base de datos no pasen desapercibidos y sean manejados de manera controlada.	Implementar registros detallados de errores (logging), mensajes más descriptivos para depuración y mecanismos de recuperación o reintento ante fallos temporales de persistencia.

Prueba	Métricas (¿qué se probó?)	Resultado esperado	Hallazgos (resultado real)	Defectos (¿qué falló?)	Importancia (¿Por qué?)	Recomendación de mejora
Pytest						
P-01 test_consultar_productos	- Código de estado HTTP igual a 200 (OK). - La respuesta es de tipo lista (array). - La lista contiene al menos un elemento.	200 OK, lista de productos con al menos un producto.	Aprobado. Se obtuvo una respuesta del API adecuada que permitió validar el contrato del endpoint.	Ninguno	Alta. Permite determinar la conexión del API, además de determinar una correcta recepción de productos.	El endpoint da una respuesta donde lista los productos correctamente.
P-02 test_insertar_producto	- Código de estado HTTP igual a 200 (OK). - Inserción exitosa del producto en la base de datos - Total de registros en el carrito igual a 1	200 Ok, inserción exitosa en la BD.	100% aprobado (3/3). Se obtuvo una respuesta de la API y el producto se insertó en la BD exitosamente.	El umbral de tiempo llegó a los 1.81 segundos.	Alta. Representa la funcionalidad principal del API puesto permite almacenar los productos en la base de datos, con ello tener persistencia en cada prueba.	Aumentar el umbral de tiempo en consultas con una gran cantidad de registros.
P-03 test_validar_total_carrito	- Respuesta exitosa del API (código HTTP 200). - Inserción correcta de los registros en la base de datos SQLite. - Cálculo correcto del total esperado de productos. - Cálculo correcto del monto total esperado. - Comparación entre los	Total de productos según lo esperado por py aprox.	100% aprobado (5/5). Resultados coinciden con los esperados	Ninguno	Alta. Es importante que el total del carrito sea consistente con el total real al sumar los productos.	La integración entre la API y la BD es exitosa y el total es el esperado.

Prueba	Métricas (¿qué se probó?)	Resultado esperado	Hallazgos (resultado real)	Defectos (¿qué falló?)	Importancia (¿Por qué?)	Recomendación de mejora
	valores esperados y los obtenidos desde la base de datos. - Validación de precisión en valores decimales mediante <code>pytest.approx</code> .					
P-04 test_validar_cantidad_negativa	- Consulta de producto válido desde la API. - Intento de inserción con cantidad negativa. - Verificación de generación de error (IntegrityError).	IntegrityError al intentar insertar cantidad = -1	100% aprobado (2/2). SQLite rechazó correctamente el valor negativo	Ninguno	Alta. Es importante atrapar con errores cantidades incorrectas negativas.	Se valida la respuesta de la API y la BD tiene una restricción exitosa.
P-05 test_insertar_producto_inexistente	- Código de estado HTTP igual a 200 - La respuesta de la API para el id inválido retorna vacío. - No se realiza ninguna inserción en la base de datos - Total de registros en el carrito igual a 0	API retorna vacío, 0 registros en BD	100% aprobado (4/4). API retornó vacío y no se insertó nada en BD	Ninguno	Alta. Es importante que no se pueda ingresar al sistema un producto inexistente o inválido.	Se valida la respuesta de la API y se manejan correctamente los productos inexistentes a nivel BD.
P-06 test_tiempo_respuesta_api_menos_1000ms	- Código de estado HTTP igual a 200 (OK) - Tiempo de respuesta inferior a 1000 milisegundos	Respuesta en menos de 1000 ms	Aprobado. Tiempo observado: 650 ms	Ninguno	Media. Es importante que el tiempo de respuesta del sistema sea razonable.	El endpoint cumple con el tiempo de respuesta mínimo.
P-07 test_contrato_	- id es de tipo int - title es de tipo str	Todos los campos cumplen con los	100% aprobado (7/7). Tipos válidos,	Ninguno	Alta. Es importante que los tipos de	La BD cumple con el contrato estricto y

Prueba	Métricas (¿qué se probó?)	Resultado esperado	Hallazgos (resultado real)	Defectos (¿qué falló?)	Importancia (¿Por qué?)	Recomendación de mejora
estricto_productos	- price es de tipo int o float - category es de tipo str - image es de tipo str - Inserción exitosa en la base - de datos sin errores de integridad - Total de registros en el carrito igual a 1	tipos del contrato	inserción exitosa en BD		datos de los productos sean consistentes y sigan con el contrato.	valida la respuesta de la API.
P-08 test_idempotencia_insercion_carrito	- Verificación de respuesta exitosa del API. - Inserción del producto en la base de datos. - Repetición de la misma operación con los mismos datos. - Conteo de registros para el mismo user_id y product_id. - Validación del número total de registros generados.	Operación idempotente: un solo registro en BD	No idempotente: se generaron 2 registros duplicados.	El sistema no controla duplicados al insertar el mismo producto	Media. Puede ser importante que no se pueda guardar dos veces el mismo producto en el sistema.	Se necesita control de duplicados para garantizar consistencia
P-09 test_body_incompleto	- La API acepta el body incompleto y retorna una respuesta no nula. - El intento de inserción en la base de datos lanza un error de integración. - Total de registros en el carrito igual a 0.	API acepta; BD lanza IntegrityError; 0 registros insertados	100% aprobado (3/3). BD rechazó correctamente la inserción incompleta	Ninguno	Alta. Es importante que el sistema reaccione correctamente con un error cuando se envía un body incompleto.	Pytest permitió validar la respuesta de la API y rechazar la inserción de body incompleto en la BD.

Prueba	Métricas (¿qué se probó?)	Resultado esperado	Hallazgos (resultado real)	Defectos (¿qué falló?)	Importancia (¿Por qué?)	Recomendación de mejora
P-10 test_integracion_parcial_api_ok_db_falla	- Respuesta HTTP correcta del API (200 OK). - Obtención válida de datos del producto. - Generación controlada del error en SQLite. - Correcta captura del error mediante Pytest. - Confirmación de que el fallo ocurre en la persistencia y no en el consumo del API.	API responde 200; BD lanza IntegrityError controlado en SQLite.	Aprobado. Se atrapó el error de manera controlada y correcta.	Ninguno	Alta. Es importante que un error no afecte el sistema completo.	El sistema maneja correctamente fallos parciales
P-11 test_mock_api_caida	- La llamada a requests.get lanza una excepción simulada. - El sistema captura correctamente la excepción - Total de registros en el carrito igual a 0 (no hubo inserción)	Excepción capturada, 0 registros en BD	100% aprobado (3/3). El mock simuló correctamente la caída de la API	Ninguno	Alta. Es importante que el sistema reaccione de manera correcta cuando la API presenta un fallo.	Pytest permite validar el comportamiento del sistema en caso de fallos de conexión sin depender del API real

Conclusiones

¿Qué diferencia observó entre una prueba de API aislada y una prueba de integración?

La principal diferencia radica en el alcance de validación.

Una prueba de API aislada evalúa un endpoint de forma individual, verificando aspectos como el estado HTTP, la estructura de la respuesta o ciertos datos específicos. Este tipo de prueba se enfoca en comprobar que el servicio responde correctamente de manera independiente.

Por otro lado, una prueba de integración evalúa la interacción entre múltiples componentes del sistema, como el consumo de la API, la lógica de negocio y la persistencia de datos. Este enfoque permite validar flujos completos y detectar errores que no serían visibles en pruebas aisladas, como inconsistencias en cálculos, problemas de almacenamiento o fallos en la comunicación entre capas del sistema.

¿Qué defectos podrían pasar desapercibidos si solo se prueba la interfaz gráfica?

Si únicamente se prueba la interfaz gráfica, varios defectos pueden no ser detectados, entre ellos:

1. Falta de validación de datos en el backend.
2. Aceptación de datos inválidos (por ejemplo, valores negativos o tipos incorrectos).
3. Respuestas incorrectas del API que la interfaz oculta o transforma.
4. Problemas en la lógica de negocio (como cálculos erróneos).
5. Manejo inadecuado de errores (respuestas sin códigos HTTP correctos).
6. Inconsistencias en la persistencia de datos.

Ya que la interfaz gráfica presenta únicamente la capa visual del sistema, sin exponer completamente el comportamiento interno ni la comunicación con el backend.

¿Qué validaciones fueron más importantes: estado HTTP, contrato JSON, reglas de negocio o persistencia? Justifique.

Todas las validaciones son importantes, pero las más críticas fueron las reglas de negocio y la persistencia.

El estado HTTP y el contrato JSON permiten verificar que la comunicación entre cliente y servidor sea correcta, asegurando que el API responde con la estructura esperada. Sin embargo, estas validaciones no garantizan que el sistema esté funcionando correctamente desde el punto de vista funcional.

Las reglas de negocio son fundamentales porque validan que el sistema cumpla con su propósito, por ejemplo, calcular correctamente el total del carrito o validar cantidades.

La persistencia es igualmente crítica, ya que asegura que los datos se almacenen de forma consistente y correcta. Sin esta validación, el sistema podría funcionar aparentemente bien, pero generar errores internos o pérdida de información.

Por lo tanto, las reglas de negocio y la persistencia tienen mayor impacto en la calidad real del sistema.

¿Qué riesgos tendría integrar un proveedor externo de pagos sin pruebas automatizadas?

Integrar un proveedor externo de pagos sin pruebas automatizadas representa riesgos significativos, tales como:

1. Procesamiento incorrecto de transacciones.
2. Cobros duplicados o incorrectos.
3. Fallos en la comunicación con el proveedor externo.
4. Inconsistencias en los montos o estados de pago.
5. Falta de detección temprana de errores.
6. Impacto directo en la experiencia del usuario y la confianza del cliente.

Además, sin pruebas automatizadas, cualquier cambio en el sistema podría introducir errores sin ser detectados oportunamente, aumentando el riesgo en producción.

¿Qué mejoraría en su estrategia de pruebas si este proyecto fuera para producción?

Si el proyecto se llevara a producción, se implementarían mejoras en la estrategia de pruebas, tales como:

1. Mayor cobertura de pruebas automatizadas (unitarias, integración y end-to-end).
2. Validaciones más estrictas de datos y reglas de negocio.
3. Implementación de pruebas de carga y rendimiento.
4. Integración con pipelines de CI/CD para ejecución automática de pruebas.
5. Uso de mocks y simulaciones para servicios externos.
6. Implementación de pruebas de seguridad.

Estas mejoras permitirían detectar errores de manera temprana, garantizar la estabilidad del sistema y reducir riesgos en ambientes productivos.